

Σavante

**Diseño y Programación
Orientada a Objetos. Patrones
de Diseño y Lenguaje de
Modelado Unificado (UML)**

ÍNDICE

1. Programación orientada a objetos	4
1.1. Estructura de un programa orientado a objetos	7
1.2. Modelado de sistemas	8
1.3. Ejemplos de POO	9
2. Elementos y componentes software	10
2.1. Mensajes	11
2.2. Objetos	11
2.3. Métodos	13
2.4. Clases	13
2.4.1. Instanciar una clase (Instancia)	16
2.5. Ejemplo	17
3. Propiedades básicas de LPOO	19
3.1. Técnicas del sexenio	19
3.1.1. Herencia	19
3.1.1.1. Generalización	20
3.1.1.2. Especialización	21
3.1.2. Abstracción	22
3.1.3. Polimorfismo	24
3.1.3.1. Sobrecarga	25
3.1.4. Acoplamiento	28
3.1.5. Cohesión	28
3.1.6. Encapsulamiento	29
3.2. Reutilización o reusabilidad de código	31
3.3. Relaciones entre clases	32
3.3.1. Asociación	33
3.3.2. Agregación	34
3.3.3. Composición	36
3.3.4. Determinar las relaciones	37
3.4. Recolección de basura	38
3.5. Caja negra	39

4. Ventajas y desventajas de la POO	39
5. Patrones de diseño (GOF)	40
5.1. Patrones GOF de Creación	42
5.2. Patrones GOF Estructurales	43
5.3. Patrones GOF de Comportamiento	45
6. GRASP (General Responsibility Assignment Software Patterns)	46
7. Modelado de aplicaciones: UML	48
7.1. Bloques de construcción de UML	52
7.1.1. Elementos	52
7.1.2. Relaciones	54
7.2. Diagramas UML	55
7.2.1. Diagramas UML Estructurales	56
7.2.2. Diagramas UML de Comportamiento	58
7.2.2.1. Diagramas de casos de uso	61
7.2.3. Manual de Modelado UML	63
8. El Proceso Racional Unificado (RUP)	65
9. Bibliografía	67



1. Programación orientada a objetos

Debido a las limitaciones de la programación estructurada, se crea la programación orientada a objetos POO. (Según sus siglas en inglés, OOP)

En el mundo real, existen con objetos (personas, coches...) que tienen atributos (datos) y comportamientos (funciones).

- **Atributos.**

Son las características o propiedades de los objetos.

Ejemplos: estatura de una persona, color de un coche, número de plantas de un edificio.

- **Comportamiento.**

Son las acciones que realizan los objetos del mundo real en respuesta a un determinado estímulo.

Ejemplo: una persona puede sentarse, caminar, conducir.... un coche puede acelerar, frenar...

La programación orientada a objetos, es un paradigma de programación que utiliza objetos y sus interacciones, innova la forma de obtener resultados, los objetos manipulan los datos de entrada para la obtención de datos de salida específicos, donde cada objeto ofrece una funcionalidad especial.

Es un conjunto de objetos que interactúan entre sí enviándose mensajes.

Este lenguaje, está basado en varias técnicas: abstracción, herencia, polimorfismo, encapsulamiento...

Su uso comenzó a principios de los años 1990 y se hizo muy popular, por lo que actualmente, existen muchos lenguajes de programación que soportan la orientación a objetos.

Los objetos tienen atributos y métodos.

Esta característica permite modelar los objetos del mundo real de un modo mucho más eficiente que utilizando funciones y datos.



Objetivos

Enfoques de los lenguajes de programación:

- En el **enfoque procedimental**, nos preguntamos:
¿Qué hace este programa?
- En el **enfoque orientado a objetos** nos preguntamos:
¿Qué objetos del mundo real podemos modelar?

Por lo tanto, en lugar de ajustar un problema al enfoque procedimental de un lenguaje, en la programación orientada a objetos intentamos ajustar el lenguaje al problema.

La POO (programación orientada a objetos):

- Se basa en el hecho de que se debe dividir el programa en modelos de los objetos físicos en lugar de en tareas.
- La idea fundamental es combinar en una sola entidad (objeto) tanto los datos como las funciones que actúan sobre los datos.
- Tiene como base averiguar que objetos necesita el programa y cuáles deben ser sus atributos y sus métodos.

Diseño

Al crear un sistema de BD Orientado a Objetos debemos tener en cuenta unas características que están divididas en tres grupos:

- **Mandatorias.** Son las características obligatorias que un sistema de bases de datos orientado a objetos debe cumplir para poder considerarse como tal.
 - **Objetos e Identidad (OID):** Cada objeto tiene un identificador único permanente, independiente de los valores de sus atributos.
 - **Encapsulación:** Los datos (atributos) y el comportamiento (métodos) se empaquetan juntos dentro de un objeto.
 - **Clases:** Las clases definen la estructura y el comportamiento común para un grupo de objetos similares.
 - **Herencia:** Las clases pueden heredar atributos y métodos de sus superclases, facilitando la reutilización.
 - **Polimorfismo y Enlace Tardío:** El mismo mensaje puede provocar comportamientos diferentes dependiendo del objeto receptor.



- **Complejidad Computacional:** El lenguaje del sistema debe ser capaz de resolver cualquier problema computable.
- **Persistencia Transparente:** Los objetos sobreviven al programa que los creó sin necesidad de conversión.
- **Opcionales.** No son estrictamente necesarias, pero su incorporación mejora el sistema al añadir funcionalidad extra o un mejor rendimiento.
 - **Herencia Múltiple:** Una clase puede heredar estructura y comportamiento de más de una superclase directa.
 - **Verificación de Tipos:** El sistema asegura la compatibilidad de tipos tanto en compilación como en ejecución.
 - **Distribución:** La base de datos y el procesamiento pueden estar repartidos en múltiples ubicaciones físicas.
 - **Transacciones:** Soporte para operaciones complejas que deben cumplir las propiedades ACID (atomicidad, consistencia, aislamiento, durabilidad).
 - **Consultas (Querying):** Capacidad de realizar búsquedas complejas y declarativas sobre los objetos persistentes.
 - **Control de Versiones:** Mecanismo para guardar, gestionar y acceder a distintas versiones de un objeto.
- **Abiertas.** Son aquellas en las que el diseñador puede aportar libertad de implementación. Están ligadas al entorno de programación y permiten extender o adaptar el sistema a necesidades específicas.
 - **Paradigma del Lenguaje:** Decisión de acoplar la base de datos a un lenguaje de programación específico o ser multi-lenguaje.
 - **Sistema de Tipos:** Diseño e implementación específica de cómo se comprueban y manejan los tipos de datos.
 - **Estrategia de Persistencia:** Elección del método para lograr la persistencia (por herencia, reachability o declaración).
 - **Gestión de Caché:** Implementación de algoritmos para manejar qué objetos se mantienen en memoria RAM.
 - **Mecanismo de Indexación:** Diseño de cómo se indexan los objetos para acelerar las consultas.

Cuando escribimos un programa en un lenguaje orientado a objetos, estamos creando un modelo de una parte del mundo real.

Las partes que construimos son objetos que aparecen en el dominio del problema.



Los lenguajes orientados a objetos:

- Utilizan atributos de una forma equivalente a las variables.
- Utilizan métodos de una forma equivalente a las funciones.



Recuerda ver las clases emitidas en Temario Audiovisual

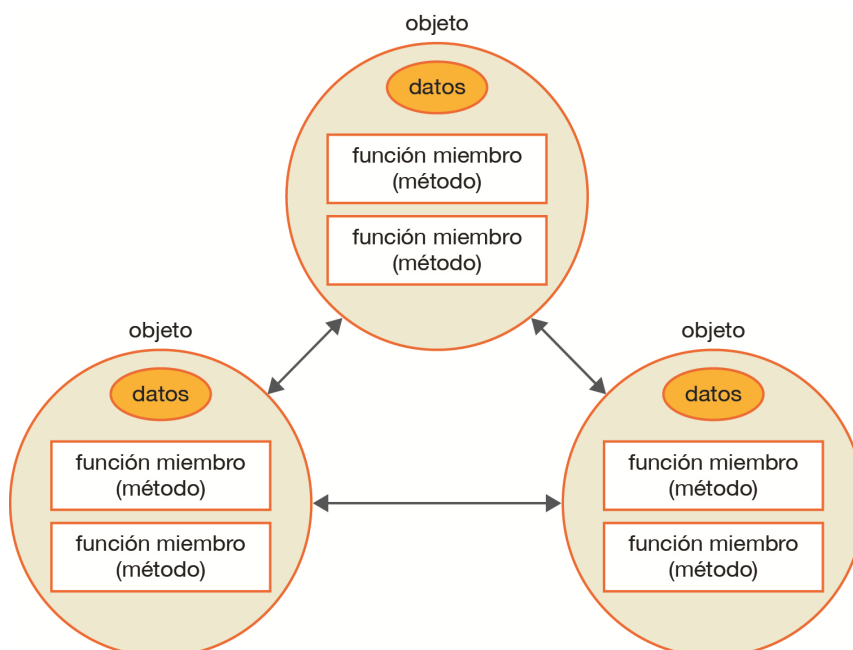
Las clases impartidas en directo y disponibles en Campus, en **Temario Audiovisual**, te ayudarán al entendimiento de la unidad, y además pueden tener información adicional.

[ACCEDE DIRECTAMENTE DESDE AQUÍ](#)

1.1. Estructura de un programa orientado a objetos

En un sistema orientado a objetos, el programa se organiza como un conjunto finito de objetos que contienen datos y métodos, (funciones miembro) que operan sobre esos datos y que se comunican entre sí mediante mensajes.

La estructura de un programa orientado a objetos sería la siguiente:





1.2. Modelado de sistemas

Las etapas necesarias para modelar un sistema y resolver un problema son:

1. Identificación de los objetos del problema.
2. Agrupamiento los objetos en clases (tipos o categorías de objetos) según sus características y comportamiento.
3. Identificación de los atributos y operaciones de cada una de las clases.
4. Identificación de las relaciones existentes entre las diferentes clases del modelo.

Identificación de objetos

Un objeto en *software* es una entidad individual de un sistema que guarda una relación directa con los objetos del mundo real. La correspondencia entre objetos de programación y objetos del mundo real es el resultado práctico de combinar atributos y operaciones.

Un objeto tiene un **estado**, un **comportamiento** y una **identidad**:

- **Estado.**

Es el conjunto de valores de todos los atributos de un objeto en un instante de tiempo determinado.

El estado de un objeto tiene un carácter dinámico que varía a lo largo del tiempo.

- **Comportamiento.**

Es el conjunto de operaciones que se pueden realizar sobre un objeto.

Las operaciones pueden ser de observación del estado interno del objeto (consultar el valor de un atributo), o bien de modificación de dicho estado (cambiar el valor de un atributo).

El estado de un objeto evoluciona en función de la aplicación de sus operaciones.

Estas operaciones se realizan tras la recepción de un mensaje o estímulo externo enviado por otro objeto.

- **Identidad.**

Permite diferenciar los objetos de modo no ambiguo. Es independiente de su estado y permite distinguir dos objetos idénticos en cuanto a los valores de sus atributos.

Cada objeto posee su propia identidad y ocupa su propia posición en la memoria de la computadora.



Cuando se diseña un problema en un lenguaje orientado a objetos, se debe pensar en dividir dicho problema en objetos. Dicho de otro modo, es preciso identificar y seleccionar los objetos del dominio del problema de modo que exista una correspondencia entre los objetos desde el punto de vista de programación y los objetos del mundo real.

El tipo de cosas que pueden convertirse en objetos es infinito.



Ejemplo

Algunos casos típicos de objetos podrían ser:

- Personas (clientes, alumnos, empleados).
- Estructuras de datos (pilas, árboles).
- Archivos de datos (fichero, almacén de datos).
- Objetos físicos (coches, muebles, juguetes).
- *Software* (S.O., procesador de textos).

Podríamos seguir y no terminaríamos nunca.

1.3. Ejemplos de POO

ADA 95	Ruby	PHP 4.0
C++	Python	Kotlin
C#	OCAML	Vala
Objective C	Object Pascal	Visual Basic 6.0
Java	CLIPS	Simula
Smalltalk	Actionscript	Delphi
SCALA	Pauscal (En español)	PowerBuilder
Eiffel	Perl	Visual FoxPro



Tengamos en cuenta:



Atención

LPOO "PURO"

Smalltalk: de los primeros LPOO con tipado dinámico. Es considerado un "mundo virtual de objetos" donde cualquier entidad es modelada como un objeto.

SCALA: LPOO puro; cada valor es un objeto. El tipo y comportamiento de los objetos se describe por medio de clases y traits. La abstracción de clases se realiza extendiendo otras clases y usando un mecanismo de composición basado en mixins como un reemplazo limpio de la herencia múltiple.

2. Elementos y componentes software

Un componente de software es un elemento de un sistema que ofrece un servicio predefinido, y es capaz de comunicarse con otros componentes.

Un componente es un objeto escrito de acuerdo a unas especificaciones, debe ser diseñado e implementado de tal forma que pueda ser reutilizado en muchos programas diferentes, adquiriendo la característica de reusabilidad.

Esta capacidad de reusabilidad (reusability), es una característica importante de los componentes de software de alta calidad. Un componente.

Requiere gran esfuerzo y atención escribir un componente que es realmente reutilizable.

El componente debe estar:

- Completamente documentado.
- Probado intensivamente.
- Debe ser robusto, comprobando la validez de las entradas.
- Debe ser capaz de pasar mensajes de error apropiados.
- Diseñado pensando en que será usado de maneras imprevistas.



2.1. Mensajes

Un mensaje es la petición que enviamos a un objeto para que se comporte de una determinada manera, es decir, que realice una acción.

En respuesta al mensaje, el objeto receptor se comportará de una determinada forma.

"El mensaje invoca a un método que realizará la acción sobre el objeto".

Sintaxis:

```
<Variable_Objeto>.<Nombre_Método> ( [<Lista de Parámetros> ] );
```

Cuando el objeto receptor recibe el mensaje, comienza la ejecución del algoritmo contenido dentro del método invocado, recibiendo y/o devolviendo los valores de los parámetros correspondientes, si los tiene ya que son opcionales: ([]).

Cada mensaje consta de tres partes:

1. Identidad del objeto al que va dirigido el mensaje.
2. Operación solicitada (método).
3. Información adicional (argumentos), necesaria para poder ejecutar el método.

Mediante el mensaje podemos:

- Activar el comportamiento de un objeto: Un mensaje desencadena la ejecución de un método en el objeto receptor.
- Establecer interacciones entre objetos: Los objetos colaboran entre sí enviándose mensajes.
- Implementar el principio de encapsulación: El mensaje es la única forma de acceder al comportamiento de un objeto, sin exponer su implementación interna.

2.2. Objetos

El objeto es el centro de la programación orientada a objetos. Un objeto es algo que se visualiza, se utiliza y que juega un papel o un rol.

Cuando se programa de modo orientado a objetos se trata de descubrir e implementar los objetos que juegan un rol en el dominio del problema del programa.

La estructura interna y el comportamiento de un objeto, en consecuencia, no son prioritarios durante el modelado del problema (abstraemos las particularidades internas del objeto).



Un objeto:

- Almacena unos valores, se denominan atributos, variables o propiedades.
- Pueden realizar acciones, que, se denominan: servicios, funciones, métodos, procedimientos u operaciones.

¿Qué puede ser un objeto?

Un objeto no tiene que ser necesariamente algo concreto o tangible. Puede ser totalmente abstracto e incluso describir un proceso.



Ejemplo

Una **asociación** de senderismo podría ser un objeto.

- Los **atributos** podrían ser el nombre, número de miembros, la ubicación de la sede.
- Los **métodos** podrían ser convocar salida, añadir miembro, dar de baja a un miembro, etcétera.
- Una **instancia** podría ser:
 - Nombre: Asociación de Senderismo Amigos del Calvario.
 - Miembros: 200.
 - Ubicación de la sede: Villafranca de Córdoba.



Fuente: <https://pxhere.com/en/photo/916422>



Los fundamentales elementos de un objeto son tres:

- Estado.
- Comportamiento: El conjunto de operaciones que se pueden realizar sobre un objeto en un momento dado.
- Identidad.

2.3. Métodos

En programación orientada a objetos, las operaciones definidas para los objetos, se denominan métodos.

Los métodos son subrutinas de manipulación de dichos datos, implementan la funcionalidad asociada al objeto.



+ Info

Los métodos, son el equivalente a las funciones en programación estructurada, con la diferencia de que es posible acceder a las variables de la clase de forma implícita o incluida.

Cuando se llama a un método de un objeto, se interpreta como el envío de un mensaje a dicho objeto.

Un programa orientado a objetos se forma enviando mensajes a los objetos, que a su vez envían mensajes a otros objetos.

2.4. Clases

Todos los objetos del mismo tipo se agrupan en clases. Una clase, es una plantilla con un modelo predefinido, para la creación de objetos de datos de determinado tipo.

Una clase es la implementación de un tipo abstracto de dato y describe no solo los atributos (datos) de un objeto, sino también sus operaciones (comportamiento).



Una clase puede existir sin que se haya creado ningún objeto a partir de ella. Los objetos dependen de la clase, pero no al revés. Una



Básico

Una clase para representar a personas puede llamarse "Humanos" y tener una serie de atributos como nombre, edad (normalmente son propiedades), y una serie de comportamientos que pueden tener, como reír, llorar, comer...y que se implementan como métodos de la clase (funciones).

Cada clase define:

- Los atributos son las características que describen a un objeto, en la práctica se implementan como variables de objeto, también llamadas variables de instancia.
- Los métodos, son las operaciones que pueden realizar los objetos de una clase (sobre sí mismos o sobre otros objetos).

Cada objeto creado a partir de la clase se denomina **instancia** de la clase.



+ Info

Los lenguajes de programación que soportan clases, pueden diferir en su soporte algunas características de las clases. La mayoría soportan diversas formas de herencia y, también características para proporcionar encapsulación, como especificadores de acceso.

Una clase puede tener una representación (meta-objeto) en tiempo de ejecución, proporcionando apoyo en tiempo de ejecución para la manipulación de los datos relacionados con la clase.

Una clase puede tener elementos privados, por tanto, cuando una clase hereda de otra, todos los elementos privados de la clase base no son accesibles a la clase derivada.



Componentes

Las clases se componen de "miembros" (elementos) de varios tipos:

- **Campos de datos:** almacenan el estado de la clase por medio de "variables o miembros".

Los datos pueden estar almacenados en variables, o en estructuras más complejas como son: uniones, structs e incluso otras clases.

(Struct: es una declaración de tipo de datos compuestos que define una lista de variables agrupadas físicamente con un solo nombre en un bloque de memoria).

Las variables miembro, normalmente, son privadas al objeto (principio de ocultación) y su acceso se realiza mediante propiedades o métodos que realizan comprobaciones adicionales.

- **Las "propiedades":** Las propiedades son los atributos de la computadora. Debido a que es común que las variables miembro sean privadas, para controlar el acceso y mantener la coherencia, surge la necesidad de permitir consultar o modificar su valor, mediante pares de métodos: GetVariable y SetVariable.
- **Métodos en las clases:** Los métodos implementan la funcionalidad asociada al objeto.

Cuando se desea realizar una acción sobre un objeto, se dice que se le manda un mensaje invocando a un método que realizará la acción.

Son el equivalente a las funciones en programación estructurada. Se diferencian de ellos en que es posible acceder a las variables de la clase de forma implícita o incluida.



Pista

Haciendo una comparación con la gramática en lenguaje, si las clases representan sustantivos:

- Los campos de datos pueden ser sustantivos o adjetivos.
- Los métodos son los verbos.



2.4.1. Instanciar una clase (Instancia)

Una instancia (en inglés, instance) es el resultado "crear" un objeto, su realización específica u ocurrencia. Cuando creamos un objeto, estamos instanciando una clase.

Una clase describe un conjunto de objetos mediante atributos y métodos que resumen sus características y comportamientos. Es una plantilla o definición de los objetos que se crearán a partir de ella.

Definir clases permite trabajar con código reutilizable.

En los lenguajes de programación orientada a objetos, podemos decir, que un objeto y la instancia de una clase son sinónimos, el matiz es que cuando hablamos de instancia nos referimos de manera velada al molde (la clase).

Las instancias son la implementación de los objetos descritos en una clase.

Estas instancias constan de atributos descritos en la clase y se pueden manipular con las operaciones definidas en la propia clase.

Cuando se declara un objeto de tipo "Automóvil", se está creando una instancia (objeto) de la clase "Automóvil".

Una clase es una entidad imprescindible que define atributos y métodos concretos que tendrá un objeto al instanciarse.

Definir clases permite encapsular datos y comportamiento en una misma abstracción. Varias instancias comparten la misma implementación de los métodos, mientras cada una mantiene su propio estado. Las instancias se crean explícitamente a partir de la clase; la reutilización del código proviene de esa implementación común y se potencia mediante composición, herencia y polimorfismo, conceptos estos últimos que veremos un poco más tarde.

Las clases definen qué atributos existen y qué operaciones posibilitan, dejando que cada objeto particular tenga sus propios valores, en lo que se definirá como estado.

Un progreso importante en la historia de los lenguajes de programación se produjo cuando se comenzó a encapsular o empaquetar diferentes propiedades en un tipo de dato.

Las estructuras de datos permiten agrupar en una sola variable varios campos relacionados. El objeto surge como evolución del struct de la programación estructurada, al incorporar no solo datos, sino también operaciones asociadas.

Sin embargo, aunque en las estructuras y registros se pueden almacenar las propiedades individuales de los objetos, no pueden representar qué hacer con estos objetos (acelerar, frenar, etcétera). Por lo tanto, se necesita que estas operaciones también se incorporen al objeto.



El **tipo abstracto de datos (TAD)** describe los atributos de un objeto y también su comportamiento (operaciones o funciones). El término tipo abstracto de dato se consigue en programación orientada a objetos con el término clase. El TAD se corresponde con la clase en POO.

2.5. Ejemplo

Recordamos

Cada clase define: (describe todos los objetos de una determinada categoría).

- Un conjunto de variables.
- El comportamiento: métodos apropiados para operar con dichos datos.

Cada objeto creado a partir de la clase se denomina instancia de la clase.

Los objetos se crean a partir de las clases:

- La clase describe el tipo del objeto.
- Los objetos representan instanciaciones individuales de la clase.

Los fundamentales elementos de un objeto son tres:

- Estado.

El conjunto de valores de los atributos en un momento determinado.

- Comportamiento.

El conjunto de operaciones (que hemos definido) que se pueden realizar sobre un objeto en un momento dado.

- Identidad.

Propiedad que permite distinguir un objeto de cualquier otro, aunque tengan el mismo estado. A nivel de implementación, suele corresponder a la referencia o dirección de memoria (o a un identificador interno) asignada por el sistema.

Ejemplo:

Un automóvil sería una clase con:

- Atributos:

Modelo, color, número de puertas, tapicería.



- Comportamientos:

Acelerar, girar, frenar.

Una **instancia** de la clase automóvil sería:

- Un coche específico, es una instancia de la clase automóvil (teniendo en cuenta el modelo, numero de puertas, color, etc.)

Una persona sería una clase con:

- Atributos.

Altura, peso, edad, etc.

- Comportamientos:

Caminar, **conducir** (acelerar, girar el volante, frenar).



Reto

Este coche, independientemente de su matrícula y número de bastidor, es único, tiene valores únicos en alguno/s de sus atributos que no tiene ningún otro coche.

¿Se te ocurre que atributos pueden ser, o el motivo?

Solución:

Es un coche tuneado, tapicería, maletero etc...

Por tanto, tiene atributos que no comparte con ninguna otra instancia "coche".

Sería un solo objeto dentro de una clase.



Fuente: imágenes cedidas por Daniel Sanz

El conductor deberá, tener el comportamiento de conducir, y enviar mensajes al objeto coche, para que acelere, gire, frene etc.



3. Propiedades básicas de LPOO

La mayoría de los lenguajes de programación actuales, incluyen bibliotecas o librerías.

También permiten al usuario la creación de sus propias bibliotecas.

3.1. Técnicas del sexenio

El lenguaje de Programación Orientado a Objetos, se basa en varias técnicas del sexenio:

- Herencia (generalización).
- Abstracción.
- Polimorfismo.
- Acoplamiento.
- Cohesión.
- Encapsulamiento.

3.1.1. Herencia

Es uno de los conceptos más importantes del paradigma orientado a objetos.

Es una abstracción que permite la reutilización de código, y, además, habilita las capacidades del polimorfismo, a través de la sobre escritura de métodos.

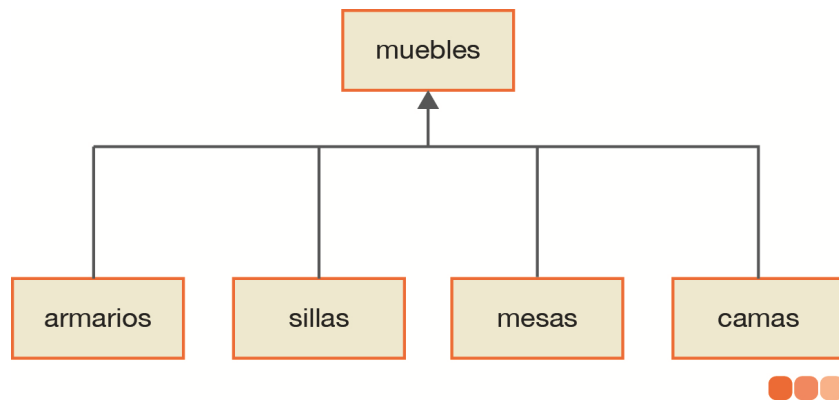
A través de ella, los diseñadores pueden crear nuevas clases partiendo de una clase o de una jerarquía de clases preexistente (ya comprobadas y verificadas) evitando con ello el rediseño, la modificación y verificación de la parte ya implementada, una clase se deriva de otra, de manera que extiende su funcionalidad.

La herencia facilita la creación de objetos a partir de otros ya existentes e implica que una subclase obtiene todo el comportamiento (métodos) y eventualmente los atributos (variables) de su superclase.

En los lenguajes que cuentan con un sistema de tipos fuerte y estrictamente restrictivo con el tipo de datos de las variables, la herencia suele ser un requisito fundamental para poder emplear el Polimorfismo, al igual que un mecanismo que permita decidir en tiempo de ejecución qué método debe invocarse en respuesta a la recepción de un mensaje, conocido como enlace tardío o enlace dinámico.

Las clases pueden dividirse en subclases.

Ejemplo: La clase mueble se puede dividir en sillas, mesas, armarios, etcétera. (todo fabricado en el mismo material, como por ejemplo madera).



Herencia simple

La clase de la que se hereda se suele denominar clase principal, clase base, clase padre, superclase o clase ancestro (el vocabulario que se utiliza suele depender en gran medida del lenguaje de programación).

Las subclases también se denominan clases derivadas.

Cada subclase comparte características con la clase de la que deriva (por ejemplo, el material del que están hechos, el tamaño, el precio, etcétera).

Además, cada subclase tendrá sus propias características (la subclase "armario" tendrá el atributo "número de puertas" y la silla el "número de patas").

Hay dos tipos de herencias:

- **Herencia simple.** Una clase sólo puede derivar de otra (solo puede tener una superclase).
- **Herencia múltiple.** Una clase deriva de dos o más clases, (puede tener más de una superclase). No todos los lenguajes POO ofrecen esta posibilidad.

Lenguajes que soportan herencia múltiple en su mayor parte son: C++, Centura SQL Windows, CLOS, Eiffel, Object REXX, Perl y Python.

Es una herramienta muy potente, pero puede producir problemas, como un conflicto de nombres cuando el mismo nombre se utiliza en dos o más clases. Por eso algunos lenguajes no la implementan.

En programación orientada a objetos, la herencia es, después de la agregación o composición, el mecanismo más utilizado para alcanzar algunos de los objetivos más preciados en el desarrollo de software como lo son la reutilización y la extensibilidad.

3.1.1.1. Generalización

La herencia modela el hecho de que estos objetos tienden a organizarse en jerarquías. Esta jerarquía, desde el punto de vista del modelado, se denomina relación de generalización y se define con el predicado "es-un".



- Ejemplo: una cama es un mueble.

Por lo tanto, la herencia es la relación de generalización.

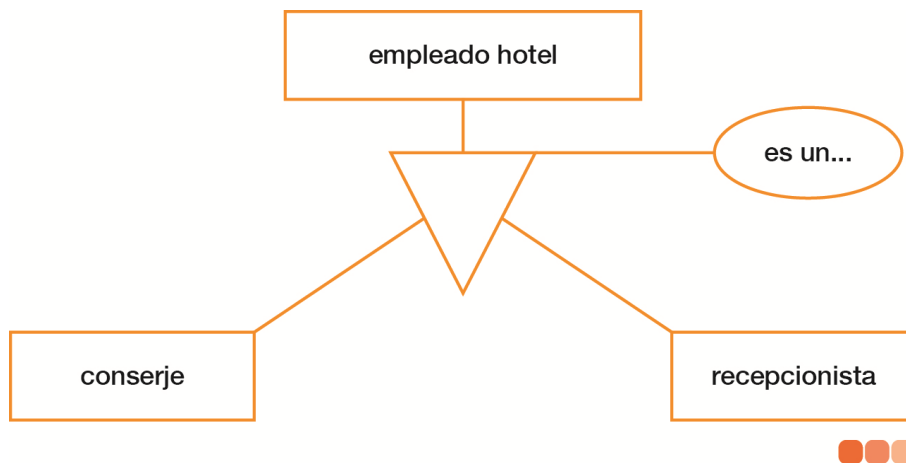
Cada clase derivada hereda las características de la clase base y cada clase derivada añade sus propias características (atributos y operaciones).

Las clases bases pueden a su vez ser también subclases o clases derivadas de otras superclases o clases base.

Una generalización es una relación de herencia entre dos clases. Permite a una clase heredar atributos y operaciones de otra clase. Su implementación en un lenguaje orientado a objetos es la herencia.

Es la "relaciónX" que existe entre una entidad y los subtipos de entidad más específicos que dependen esa "relaciónX". Se representa mediante un triángulo invertido.

Ejemplo: los tipos conserje y recepcionista obteniendo el supertipo empleado.



La herencia nos permite abstraer un tipo de entidad de nivel superior (supertipo) a partir de varios tipos de entidad (subtipos); en estos casos los atributos comunes y relaciones de los subtipos se asignan al supertipo.

3.1.1.2. Especialización

Es la relación opuesta a la generalización. Se puede definir con la relación "es un". Esta relación es transitiva.



Ejemplo

Perro:

- Un **perro** es un **canino**.
- Un **canino** es un **mamífero**.

Como esta relación es transitiva, tenemos que:

Un perro es un mamífero.



Nuestra perrita Kora es un mamífero

3.1.2. Abstracción

La abstracción es la propiedad que considera los aspectos más significativos o notables de un problema y expresa una solución en esos términos, omitiendo la información no relevante para simplificar el problema.

La abstracción se representa con el diseño de una clase que implementa la interfaz correspondiente. La abstracción posee diversos grados, denominados niveles de abstracción, que ayudan a estructurar la complejidad intrínseca que poseen los sistemas del mundo real.

En el análisis de un sistema hay que concentrarse en ¿qué hace? y no en ¿cómo lo hace?



Ejemplo

Un objeto podría ser una consola de videojuegos.

Conocemos el modelo (PS4 o XBOX ONE), su precio y otros atributos, y también operaciones como apagar, encender, conectar periféricos, etcétera.

Sin embargo, para hacer uso de ella no necesitamos conocer cómo funciona internamente.

Lo mismo pasaría con un juego. Sabemos jugar al SKYRIM (¡qué pedazo de juego!) y cómo interactuar con él por medio del mando, pero no necesitamos saber cómo se ha programado para disfrutarlo.



Fuente: <https://pxhere.com/en/photo/933936>

En programación se utiliza la abstracción para definir funciones o clases. Por ejemplo, podríamos utilizar la función para el cálculo del coseno.

Siempre que la llamada a la función y los parámetros no cambien, no nos importa cómo se haya codificado internamente. Incluso podríamos modificarla y no tendría repercusión para los programas y usuarios que la utilizan (siempre que se mantenga la misma interfaz).



Bridge

El patrón **Bridge**, también conocido como **Handle/Body**, es una técnica usada en programación para desacoplar una abstracción de su implementación, de manera que ambas puedan ser modificadas independientemente sin necesidad de alterar por ello la otra.

Esto es, se desacopla una abstracción de su implementación para que puedan variar independientemente.

Aplicabilidad, se usa el patrón Bridge cuando:

- Se desea evitar un enlace permanente entre la abstracción y su implementación. Esto puede ser debido a que la implementación debe ser seleccionada o cambiada en tiempo de ejecución.
- Tanto las abstracciones como sus implementaciones deben ser extensibles por medio de subclases. En este caso, el patrón Bridge permite combinar abstracciones e implementaciones diferentes y extenderlas independientemente.
- Cambios en la implementación de una abstracción no deben impactar en los clientes, es decir, su código no debe tener que ser recompilado.
- Se desea compartir una implementación entre múltiples objetos (quizá usando contadores), y este hecho debe ser escondido a los clientes.

3.1.3. Polimorfismo

Un objeto puede presentar diferentes comportamientos.

El poliformismo, es la propiedad por la cual un mismo mensaje puede originar conductas completamente diferentes al ser recibido por diferentes objetos.

Es la capacidad de una operación, de ser interpretada por el propio objeto que lo invoca, es decir, permite a una operación (método - función), tener el mismo nombre en clases diferentes, y que **actúe de modo diferente** en cada una de ellas.

Podemos enviar mensajes sintácticamente iguales a través de referencias de tipo base o interfaz. El único requisito que deben cumplir los objetos a los que se apunta es tener una implementación del mensaje que se les envía.

Esto es común en el mundo real, ya que una misma operación se realiza de forma diferente dependiendo del objeto al que se aplique.



Ejemplo

El método encender puede actuar de muchas formas diferentes según el objeto:

- Encender un ordenador.
- Encender una cerilla.
- Encender una lámpara.
- Encender un cigarro.
- Encender un coche.
- Encender los ánimos.

La apariencia del código puede ser muy diferente dependiendo del lenguaje que se utilice, más allá de las obvias diferencias sintácticas.

En lenguajes de tipado dinámico como Smalltalk, Python o Ruby, el polimorfismo se basa en el duck typing: dos objetos pueden usarse de manera polimórfica si responden al mismo mensaje (método), sin necesidad de pertenecer a una jerarquía de clases común. El sistema verifica en tiempo de ejecución si el objeto entiende el mensaje.

3.1.3.1. Sobrecarga

La sobrecarga en LPOO es un tipo de polimorfismo estático (resuelto en tiempo de compilación) que permite definir múltiples versiones de una función o operador con el mismo nombre, pero con parámetros diferentes (en tipo, número o orden).

Es la posibilidad de tener dos o más funciones con el mismo nombre, pero funcionalidad diferente. Es decir, dos o más funciones con el mismo nombre que realizan acciones diferentes. El compilador usará una u otra dependiendo de los parámetros usados. A esto se llama también sobrecarga de funciones o funciones sobrecargadas.

- Sobrecarga de métodos: En una clase, puedes tener varios métodos con el mismo nombre pero distintas firmas. El compilador elige cuál ejecutar según los argumentos.
- Sobrecarga de operadores: Algunos lenguajes (como C++ o Python) permiten redefinir operadores (ej: +, -) para que se comporten de forma distinta con diferentes tipos de datos.

Usamos los operadores o funciones de forma diferente dependiendo de los objetos sobre los que actúa.



Ejemplos sobrecarga de funciones

Vamos a ver algunos ejemplos, ya que este concepto puede ser difícil de ver si estás acostumbrado a la programación estructurada.

En el ejemplo a continuación aunque ambos métodos difieren en el tipo de retorno y en los argumentos, **el compilador solo considera las diferencias en los argumentos** para elegir cuál ejecutar.

- **Ejemplo 1:** imaginemos que tenemos un método que suma dos números y devuelve el resultado.

```
Entero suma (entero A, entero B)
{
    suma = A + B
    Return suma
}
```

¿Qué pasaría si queremos sumar dos números con decimales (*float*)?

Podemos sobrecargar el método creando otro con el mismo número, pero distintos parámetros.

```
float suma (float A, float B)
{
    suma = A + B
    Return suma
}
```

- **Ejemplo 2:** de igual forma, podríamos crear un método para inicializar una variable.

```
// Método 1: Con parámetro
entero inicializarA(entero A) {
    return A; // Devuelve el valor recibido
}
// Método 2: Sin parámetros (sobrecarga)
entero inicializarA() {
    return 0; // Devuelve 0 por defecto
}
```

En este caso tenemos dos versiones de la misma función. Si se lanza con un parámetro, se utilizará ese valor. Si no se especifica ningún parámetro, devolverá 0.



Ejemplo sobrecarga de operadores

Imaginemos dos clases: Metros y Centimetros, cada una con un atributo cantidad. Si intentamos sumar un objeto de cada clase sin sobrecargar el operador +, el compilador dará un error porque no sabe cómo realizar la operación.

Para solucionarlo, podemos sobrecargar el operador + para que convierta los centímetros a metros (dividiendo entre 100) y luego sume las cantidades.



+ Info

Vamos a simplificar mucho el código, dado que es posible que no tengas conocimientos de programación en C++ o JAVA.

Intentaremos poner lo justo, aunque no sea del todo correcto. Lo importante es entender la idea.

"Metros" es una clase que contiene un atributo llamado "Cantidad", el cual almacena la medida en metros.

"Centímetros" es una clase que contiene un atributo llamado "Cantidad", el cual almacena la medida en centímetros.

Recibimos como parámetros dos parámetros (uno de cada clase). Vamos a sobrecargar el operador "+" para que se elija la versión adecuada.

```
// Sobrecarga 1: Metros + Centimetros
Metros operator+(const Metros& m, const Centimetros& cm) {
    return Metros(m.cantidad + (cm.cantidad / 100.0f));
}

// Sobrecarga 2: Centimetros + Metros
Centimetros operator+(const Centimetros& cm, const Metros& m) {
    return Centimetros(cm.cantidad + (m.cantidad * 100.0f));
}

// Sobrecarga 3: Metros + Metros
Metros operator+(const Metros& m1, const Metros& m2) {
    return Metros(m1.cantidad + m2.cantidad);
}
```



```
// Sobrecarga 4: Centimetros + Centimetros
Centimetros operator+(const Centimetros& cm1, const Centimetros& cm2) {
    return Centimetros(cm1.cantidad + cm2.cantidad);
}
```

El compilador elige la versión correcta:

```
Metros m(2.0f);
Centimetros cm(50.0f);

Metros r1 = m + cm; // Usa sobrecarga 1 (Metros + Centimetros)
Centimetros r2 = cm + m; // Usa sobrecarga 2 (Centimetros + Metros)
Metros r3 = m + m; // Usa sobrecarga 3 (Metros + Metros)
Centimetros r4 = cm + cm; // Usa sobrecarga 4 (Centimetros + Centimetros)
```

3.1.4. Acoplamiento

Grado de interdependencia entre los distintos módulos de un programa.

Es la forma y nivel de interdependencia entre módulos, una medida de qué tan cercanamente conectados están dos rutinas o módulos del software (programa), así como el grado de fuerza de la relación entre módulos.

Un ejemplo simple de acoplamiento es cuando un componente accede directamente a un dato que pertenece a otro componente. En ese caso, el resultado del comportamiento del componente A dependerá del valor del componente B, por lo tanto, están acoplados.

El acoplamiento mide el grado de dependencia entre módulos.

El bajo acoplamiento es frecuentemente una señal de un sistema bien estructurado y de un buen diseño de software.

3.1.5. Cohesión

La cohesión tiene que ver con que cada módulo del sistema se refiera a un único proceso o entidad.

A mayor cohesión mejor: el módulo será más sencillo de diseñar, programar, probar y mantener.



La cohesión se refiere a lo que ocurre dentro de un módulo: mide si sus elementos (funciones, atributos, responsabilidades) están relacionados entre sí y contribuyen a un mismo propósito.

Cohesión significa que un módulo hace una sola cosa y la hace bien:

- Un módulo con alta cohesión tiene responsabilidades claras y bien delimitadas.
- Un módulo con baja cohesión mezcla tareas distintas o inconexas.

La meta principal del diseño orientado a objetos es alcanzar una alta cohesión modular y un bajo acoplamiento. Que un componente o módulo tenga una alta cohesión significa que es una clase bien definida, cuya responsabilidad ha sido esencializada en comportamiento y estado. ¿Significa esto que sea independiente del resto de componentes? No necesariamente.

Precisamente eso es lo que persigue la POO, aunque no exista una relación directamente causal. Ahora bien, si los componentes han sido diseñados bajo esta perspectiva y cada uno tiene una responsabilidad bien definida, nuestro sistema gozará de un alto grado de cohesión y del menor acoplamiento que hayamos sido capaces de lograr.

3.1.6. Encapsulamiento

El encapsulamiento permite aumentar la cohesión (diseño estructurado) de los componentes del sistema. Algunos autores confunden encapsulamiento con el principio de ocultación, porque se suelen emplear conjuntamente.

La encapsulación o encapsulamiento consiste en reunir, en una cierta estructura, todos los elementos que a un cierto nivel de abstracción se pueden considerar pertenecientes a una misma entidad.

Los objetos, tienen un gran sentido de la privacidad, por lo que sólo dan información sobre sí mismos, a través de los métodos que poseen para compartir su información.

Los objetos, también ocultan la implementación de sus procedimientos, aunque podemos pedirles, de un modo sencillo, mediante un mensaje, que los ejecuten.

Los usuarios y programas de aplicación, no pueden ver qué hay dentro de los métodos, solo pueden ver los resultados de su ejecución.

La idea fundamental de los LPOO, es encapsular (combinar), en una única unidad (objeto) tanto los datos como las funciones que los manipulan.

Cada objeto presenta una interface publica al resto de objetos que pueden utilizarlo.

Esta característica de encapsular, permite modelar los objetos del mundo real de un modo mucho más eficiente que con funciones y datos. Mientras que la interface publica sea la misma, se puede cambiar la implementación de los métodos sin que sea necesario informar al resto de objetos que los utilizan.



Recuerda

Los objetos que poseen las mismas características y comportamiento se agrupan en clases que no son más que unidades de programación que encapsulan datos y operaciones.

Ocultación de datos

La encapsulación, oculta lo que hace un objeto de lo que hacen otros objetos (y lo oculta del mundo exterior), por lo que se denomina también ocultación de datos. Un objeto tiene que presentar una *cara* al mundo exterior, de modo que se puedan iniciar esas operaciones.

Los usuarios de un componente solo necesitan saber qué servicios ofrece el objeto y no cómo lo hace. Por lo tanto, la interfaz pública establece qué se puede hacer con el objeto y la clase actúa como una *caja negra*.

La ocultación de datos está basada en el concepto de abstracción.

Ocultación de datos en POO:

- Los objetos se comunican mediante mensajes.
- Los mensajes son llamadas a los métodos de otro objeto.
- Para leer o modificar un objeto hay que hacer una llamada a un método del objeto para que acceda al dato y nos devuelva el valor (no se puede hacer a los datos directamente).
- Las funciones de un objeto se llaman funciones miembro o métodos (según el lenguaje de programación), y son único medio para acceder a los datos de un objeto.
- Esta propiedad facilita la escritura, depuración y mantenimiento de un programa.

Modificadores de acceso

Para lograr el encapsulamiento es necesario utilizar los modificadores de acceso asignando a los miembros (métodos o atributos) del objeto un tipo concreto visibilidad, tenemos fundamentalmente cuatro modificadores de acceso distintos:

- **`public`**: no hay encapsulamiento pues es accesible desde cualquier punto sin restricciones.
- **`private`**: solo otorga el acceso desde dentro de la misma clase.



- **`protected`**: el acceso será posible desde la propia clase, clases del mismo paquete y subclases.
- **`default`** (sin modificador): será accesible desde la misma clase y clases del mismo paquete.

La idea en LPOO es que los atributos y ciertos métodos de funcionamiento internos sean privados, proporcionando métodos públicos para acceder y modificar los atributos. Los métodos encargados de esto son los llamados *getters* o *setters*.



Resumiendo

Un objeto contiene:

- Atributos o variables de instancia (datos).
- Métodos o funciones miembro (funciones).
 - Los objetos se comunican mediante mensajes. Un mensaje es una llamada a un método del objeto.

Para pedirle datos al objeto, hay que enviarle un mensaje.

3.2. Reutilización o reusabilidad de código

Este concepto indica que una clase creada por un programador se pone a disposición de otros programadores para su utilización. También implicaría el uso de la clase por el creador para otro producto *software* diferente.

Es el mismo concepto que el de las bibliotecas de funciones de los lenguajes estructurados.

En lenguajes como C++ o Java, la reusabilidad puede ir más allá:

- Podemos coger una clase existente y crear una nueva clase que derive de esta. Heredará todas las propiedades de la clase de la que deriva, pero, además, podremos añadir nuevas características.

La facilidad de reutilizar o reusar el *software* existente es uno de los grandes beneficios de la POO.

De este modo, en una empresa de *software* se pueden reutilizar clases diseñadas en un proyecto para un nuevo proyecto, con la consiguiente mejora de la productividad, al sacarle partido a la inversión realizada en el diseño de la clase primitiva.



Ventajas de la reutilización del software

- Reducción de código.
- Solo necesitan implementarse una vez.
- Si se tienen que modificar por algún fallo, tan solo se tiene que modificar una vez (y el cambio se reflejará en todos los productos que la utilicen).
- Menor tiempo de desarrollo y menor coste de recursos.



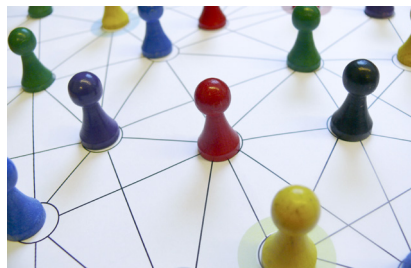
El experto opina

Cuando nos ponemos a programar, muchas veces empezamos a crear código sin investigar primero si ya existe alguna función similar a nuestra disposición, por lo que perdemos mucho tiempo y recursos *reinventando la rueda*.

Por otro lado, cuando escribas código, hazlo pensando en que pueda ser reutilizado. Te llevará algo más de tiempo, pero te lo podría ahorrar en el futuro.

Y muy importante... ¡documenta!

3.3. Relaciones entre clases



Existen cuatro tipos básicos de relación entre clases:

- Asociación.
- Agregación.
- Composición.

(Ya la has estudiado en las propiedades de LPOO).



3.3.1. Asociación

La abstracción de **asociación** nos permite vincular o asociar dos entidades independientes.

Una **asociación** queda identificada por la identificación de las entidades participantes, los roles que juegan en la relación, la multiplicidad y la navegabilidad.

Una asociación es una conexión semántica entre clases, permite que una clase conozca de los atributos y operaciones públicas de otra clase.

La asociación se podría definir como el momento en que dos objetos se unen para trabajar juntos y, así, alcanzar una meta. En este caso, ambos objetos son independientes entre sí.

La asociación representa la **relación "usa un", "tiene un" o "colabora con"**.



Refuerzo

Con la **asociación**, relacionamos dos tipos de entidades que normalmente son de dominios independientes, pero circunstancialmente (coyunturalmente) se asocian.

Se representa uniendo las entidades que participan en la asociación, unidas por una línea continua, con opcionalmente roles, multiplicidad y nombre de la asociación.

Ejemplo: una persona "usa un" gafas.

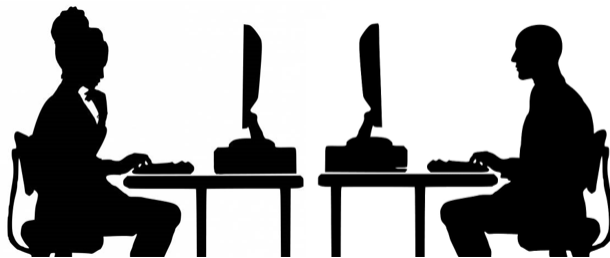




Ejemplo

Un **programador** usa un **ordenador**.

Es una relación de asociación entre la clase programador y la clase ordenador.



Fuente: PxHere y Pxfuel

3.3.2. Agregación

La abstracción de agregación nos permite construir entidades de un nivel más alto (objetos compuestos) a partir de sus entidades de nivel menor (objetos componentes).

Indica que una clase es parte de otra clase (composición débil). Los componentes pueden ser compartidos por varios compuestos (de la misma asociación de agregación o de varias asociaciones de agregación distintas).

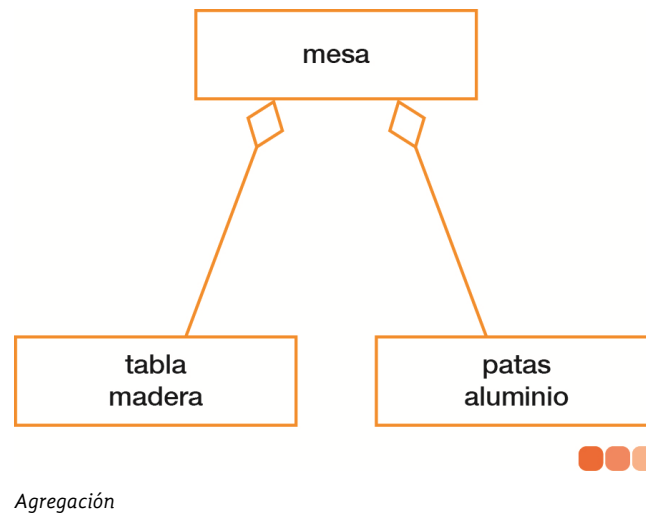
La destrucción de la agregación, no supone la destrucción de los componentes.

Permite combinar varias entidades, entre las que existe una interrelación, y así formar una entidad de más alto nivel (nivel superior). Resulta útil cuando la entidad obtenida de más alto nivel se tiene que interrelacionar con otra entidad.

Es un tipo de relación dinámica, donde, el tiempo de vida de una o más entidades de bajo nivel, que están incluidas en una entidad de alto nivel, es independiente a la entidad que la incluye (entidad de alto nivel).

La agregación en UML se representa mediante una línea de asociación entre clases, con un **rombo vacío** en el extremo de la clase que representa el todo. El otro extremo conecta con la clase parte. El rombo indica la relación de agregación (todo–parte), donde las partes pueden existir independientemente del todo.

Ejemplo: Agregamos dos entidades, en la entidad superior mesa que puede tener relaciones o incidencias con otras entidades.



Una agregación es una relación más fuerte que una asociación. Representa una clase que se compone de otras clases. La agregación es un tipo de relación dependiente.

Una agregación representa la relación (todo-parte), es decir, **una clase es el todo y contiene a todas las partes**. También se puede definir con **relación "tiene un"**.



Ejemplo

- Un **ordenador** tiene un **teclado**.
- Un **ordenador** tiene una **pantalla**.
- La clase ordenador se compone de las clases teclado y pantalla (entre otras).



Fuente: Wikipedia



3.3.3. Composición

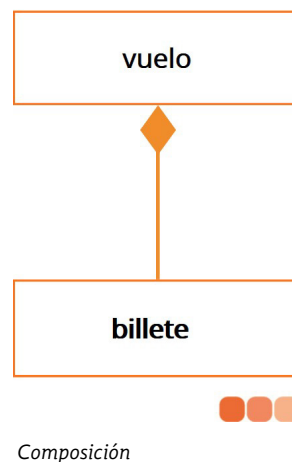
La composición es un tipo de asociación que representa una relación de contención fuerte entre una clase compuesta (el "todo") y sus componentes (las "partes"). Se caracteriza por ser una relación de pertenencia exclusiva y dependencia vital.

El ciclo de vida de los componentes está irrevocablemente ligado al ciclo de vida del objeto compuesto. Cuando la entidad de alto nivel es destruida, todos sus componentes son automáticamente destruidos con ella. Esta destrucción en cascada es inherente a la relación de composición.

En una composición, una parte (como un corazón) solo puede pertenecer a un único todo (un cuerpo) y a nadie más.

Se representa con un rombo relleno en el extremo de la clase contenedora, nunca en la clase *parte*.

Ejemplo: un billete existe en base al vuelo para el que fue emitido. Si el vuelo se anula o desaparece, automáticamente el billete también.



Diferencia entre Asociación y Composición

La diferencia fundamental entre estos tipos de relación:

- **Asociación:** cuando se elimina el todo, las entidades participantes continúan existiendo de forma **independiente**.
- **Composición:** si se elimina la entidad compuesta, desaparecen asimismo las entidades que la forman.



Multiplicidades y rangos en UML

Las relaciones entre clases en UML se definen mediante multiplicidades, que indican el número de instancias de una clase que pueden vincularse con una única instancia de la otra clase. Para ello se utilizan números, asteriscos o rangos.

- **Relación Uno a Uno (1:1 o 1):** la notación consiste en poner 1 en ambos extremos de la asociación (**1 .. 1**). Una instancia de la clase A se asocia exactamente con una instancia de la clase B y viceversa. Por ejemplo un ciudadano con un DNI.
- **Relación Uno a Muchos (1:N o 1:*):** la notación correcta es un 1 en un extremo y **0..*** (cero o más) o **1..*** (uno o más) en el otro, el asterisco significaría muchos o 0 o más. Indica que la instancia de una clase está asociada con cero o varias instancias de otra clase, pero en recorrido inverso solo está asociada con una de la primera. Tendríamos 1 en un extremo y en el otro **0..***. Por ejemplo un cliente puede hacer muchos pedidos, pero esos pedidos solo están asociados con ese cliente.
- **Relación Muchos a Muchos (N:M):** la notación correspondiente es ***..*** o simplemente *****. instancia de una clase está asociada con 0 o más instancias de otra clase y viceversa (**0..N o 0..***). Un buen ejemplo sería el de un músico que puede tocar en varias orquestas y una orquesta está compuesta de varios músicos.

3.3.4. Determinar las relaciones

Las reglas de negocio son la clave para definir la relación correcta (asociación, agregación o composición) entre las clases de un sistema. La decisión no es arbitraria; depende enteramente del contexto y la funcionalidad principal que se esté modelando.

Identificar el Todo del sistema

El primer paso es identificar el "todo", la entidad o clase principal que actúa como un contenedor para otras. Luego, se analiza cómo se relacionan las "partes" con ese "todo" para determinar la naturaleza de la dependencia.

Determinar las Relaciones de Dependencia

A continuación, vemos tres ejemplos con un escenario similar (Vuelo y Pasajero), pero con matices en las reglas de negocio que cambian la relación.

- **AGREGACIÓN: El vuelo comercial.**
 - Si el sistema se centra en la gestión de vuelos y se desea modelar una relación directa, la relación entre Vuelo y Pasajero podría considerarse una agregación (rombo vacío). El vuelo actúa como un contenedor de pasajeros, pero ellos existen de forma independiente: si el vuelo se cancela, los pasajeros siguen existiendo en el sistema.



- Si el vuelo se cancela, los pasajeros siguen existiendo en el sistema. Las partes no desaparecen si el todo lo hace.

- **Limitación importante:**

Este enfoque sería válido solo si no fuera necesario almacenar datos sobre la participación específica de un pasajero en un vuelo (como su número de asiento, clase, o fecha de embarque). La agregación es una relación estructural, pero no permite atributos.

Por ello, en la práctica, la solución más común y robusta es evitar la relación directa y utilizar en su lugar una clase de asociación (como Reserva o Billeto), que sí puede contener esos atributos.

- **COMPOSICIÓN: El vuelo Chárter.**

- En un sistema de gestión de vuelos chárter, el **Vuelo Chárter** es el "todo". Aquí, la existencia de las partes (Grupo de Pasajeros) está totalmente ligada a la existencia del vuelo.
- Si el vuelo se anula, el concepto de "ese grupo específico de pasajeros para ese vuelo" deja de tener sentido en el sistema. Si el todo desaparece, las partes lo hacen con él.

- **ASOCIACIÓN: La reserva.**

- Se utiliza cuando el sujeto del sistema es una tercera clase, como Reserva, que asocia un pasajero con un vuelo.
- La relación es simplemente una conexión lógica, sin una dependencia de todo-parte. Si una reserva se anula, vuelo y pasajero pueden seguir existiendo. El pasajero quizás haya adelantado su vuelo, etc.

3.4. Recolección de basura

La recolección de basura (garbage collection) es la técnica por la cual el entorno de objetos se encarga de destruir automáticamente, y por tanto desvincular la memoria asociada, los objetos que hayan quedado sin ninguna referencia a ellos.

Esto significa que el programador no debe preocuparse por la asignación o liberación de memoria, ya que el entorno la asignará al crear un nuevo objeto y la liberará cuando nadie lo esté usando.

En la mayoría de los lenguajes híbridos que se extendieron para soportar el Paradigma de Programación Orientada a Objetos como C++ u Object Pascal, no existe un recolector de basura nativo y la memoria debe liberarse manualmente.



3.5. Caja negra

Una caja negra es un elemento que se estudia desde el punto de vista de las entradas que recibe y las salidas o respuestas que produce, sin tener en cuenta su funcionamiento interno.

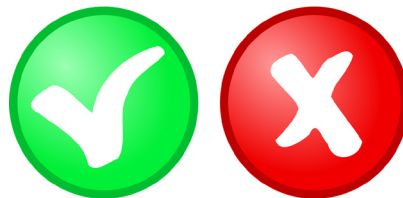
Nos interesará su forma de interactuar con el medio que le rodea (en ocasiones, otros elementos que también podrían ser cajas negras), **nos importa qué es lo que hace**, pero no nos importa **cómo lo hace**.

Debe estar muy bien definida su interfaz: sus entradas y salidas.

En programación modular, en la fase de diseño se buscará que cada módulo sea una caja negra dentro del sistema global que es el programa que se pretende desarrollar.

De esta manera se consigue una independencia entre los módulos, que facilita que el desarrollador de un módulo concreto, va a encargarse de implementar su módulo y deberá conocer como es la comunicación con los otros módulos (la interfaz), pero no necesitará conocer cómo trabajan esos otros módulos internamente, que serán para él cajas negras. Esta independencia es muy importante en los equipos de trabajo de desarrollo de un programa.

4. Ventajas y desventajas de la POO



Ventajas

- **Encapsulamiento:** Oculta los datos internos, exponiendo solo métodos controlados. Mayor seguridad y consistencia.
- **Herencia:** Reduce código redundante al permitir que clases hijas hereden y extiendan funcionalidades existentes.
- **Polimorfismo:** Objetos de diferentes clases responden al mismo método de forma específica. Flexibilidad y extensibilidad.
- **Abstracción:** Simplifica la complejidad ocultando detalles internos. Modelado más intuitivo y claro.
- **Modularidad:** Clases independientes con baja dependencia (bajo acoplamiento) y responsabilidades bien definidas (alta cohesión).



- **Reutilización:** Código fácil de extender y reusar mediante herencia/composición, acelerando el desarrollo.
- **Mantenimiento:** Estructura organizada que facilita modificaciones y reduce errores.

Desventajas

- **Curva de aprendizaje:** Es compleja para programadores acostumbrados a paradigmas estructurados, debido a su abstracción y conceptos jerárquicos.
- **Sobrecarga de documentación:** La estructura en clases y herencias requiere más documentación para clarificar relaciones y responsabilidades.
- **Interpretación subjetiva:** La abstracción de objetos puede llevar a diseños inconsistentes si no hay guidelines claros.
- **Rendimiento:** Puede implicar mayor consumo de recursos y, en algunos casos, menor velocidad debido a la gestión de objetos y al dinamismo (por ejemplo, el polimorfismo). Pese a ser irrelevante en aplicaciones modernas, es crítica en sistemas de tiempo real o bajo nivel.
- **Herencia mal diseñada:** Si la herencia no se ha diseñado correctamente, puede heredarse código innecesario o inapropiado, aumentando la complejidad y acoplamiento.
- **Verborrea de código:** Tareas simples pueden requerir más líneas de código (creación de clases, métodos) frente a soluciones procedurales.

5. Patrones de diseño (GOF)

Un patrón de diseño (design pattern), es una técnica que se utiliza para resolver problemas que ocurren frecuentemente, por ello son muy efectivos al haber sido empleado muchas veces para resolver un tipo de problema. No son fragmentos de código específicos, sino soluciones conceptuales reutilizables que deben adaptarse al contexto. Puede ser una solución a un problema de diseño.

Estos problemas se dan en el desarrollo de software, especialmente en el diseño de sistemas orientados a objetos.

El concepto de patrón de diseño comenzó a finales de los años 70, pero se hizo popular en 1994 con la publicación del libro «Design Patterns-Elements of Reusable Software» (21/10/1994), que, escrito por 4 autores (Erich Gamma, Richard Helm, Ralph Jonson y John Vlissides), describe 23 patrones de diseño como una forma indispensable de enfrentarse a la programación, que son conocidos como GoF, siglas de Gang of Four (patrones de la pandilla de los cuatro).



+Info

Eric J. Braude, indica en su libro « Ingeniería de software: una perspectiva orientada a objetos» (2003), que el objetivo de usar estos patrones, es que los cambios en los requisitos de una aplicación no provoquen modificaciones disruptivas en la estructura del sistema evitando así que afecten a las relaciones entre los objetos.

Podemos ver en un patrón de diseño 4 elementos:

- **Nombre del patrón:** Describe el problema de diseño, su solución y consecuencias. Proporciona un vocabulario común para los diseñadores.
- **El problema:** Indica el problema y su contexto, y cuando aplicar el patrón, especialmente en situaciones donde se necesite representar estructuras de clases o algoritmos como objetos.
- **La solución:** Describe la estructura de clases y objetos, sus responsabilidades y colaboraciones, de manera abstracta.
- **Las consecuencias de su aplicación:** Al aplicar un patrón habrá unos beneficios al igual que costes que será necesario evaluar para decidir las alternativas de diseño que solucionen el problema origen.

Características de los patrones de diseño:

- Son **soluciones técnicas** que proponen soluciones concretas a problemas concretos.
- Debe ser **reutilizable**, para poder ser utilizado para afrontar diferentes problemas de diseño y en diferentes circunstancias, desarrollando códigos y construyendo clases reutilizables.
- Se debe haber comprobado su **efectividad** resolviendo problemas similares en ocasiones anteriores.
- Se pretende **estandarizar** la forma de realizar un diseño de software, formalizando un vocabulario que sea común entre los diseñadores.
- Con su uso se evita la búsqueda reiterada de soluciones a problemas ya conocidos y solucionados con anterioridad.
- Su objetivo es la reusabilidad y la mantenibilidad del software.



Existen 3 tipos de patrones, en base a su finalidad:

- **De Creación:** Abordan procesos de creación de objetos, haciendo el sistema independiente de cómo se crean, componen o representan sus objetos.
- **Estructurales:** Tratan la composición de clases y objetos para formar estructuras más grandes y complejas.
- **De comportamiento:** se centran en la comunicación entre objetos y en cómo se distribuyen las responsabilidades dentro del sistema.

A continuación, explicamos cada uno de ellos.

5.1. Patrones GOF de Creación

Son aquellos que solucionan problemas de creación de objetos. Nos ayudan a encapsular y abstraer dicha creación. Puede existir más de un patrón que ayude en un mismo problema.

Los patrones de creación son:

- **Abstract Factory** (Fábrica Abstracta):

Permite trabajar con objetos de distintas familias de manera que las familias no se mezclen entre sí y haciendo transparente el tipo de familia concreta que se esté usando.

Se utiliza para crear diferentes familias de objetos, como, por ejemplo, la creación de interfaces gráficas de distintos tipos (ventana, menú, botón, etc.).

- **Builder** (Constructor virtual):

Abstrae el proceso de creación de un objeto complejo, centralizando dicho proceso en un único punto:

- Evita constructores imposibles de leer.
- Separa construcción de representación.

- **Factory Method** (Método de fabricación):

Este patrón, centraliza en una clase constructora la creación de objetos de un subtipo de un tipo determinado, ocultando al usuario la diversidad de casos particulares que se pueden prever, para elegir el subtipo que crear (esta diversidad se denomina casuística).

Se basan en que las subclases determinan la clase a implementar.

- **Prototype** (Prototipo):

Crea nuevos objetos clonándolos de una instancia ya existente.



- **Singleton** (Instancia única):

Debe garantizar, la existencia de una única instancia para una clase y la creación de un mecanismo de acceso global a dicha instancia.

Es decir, restringe la instanciación de una clase o valor de un tipo a un solo objeto.

5.2. Patrones GOF Estructurales

Son los patrones de diseño software que solucionan problemas de composición (agregación) de clases y objetos.

Son los que plantean las relaciones entre clases, las combinan y forman estructuras mayores.

Son los siguientes:

Patrón Adapter o Wrapper (Adaptador o Envoltorio)

El patrón de diseño Adapter, también conocido como Wrapper, permite que interfaces incompatibles trabajen juntas al actuar como un intermediario entre ellas. Su propósito es convertir la interfaz de una clase existente en otra que el cliente espera, facilitando la reutilización de código sin modificar su implementación original. Este patrón es particularmente útil cuando se necesita integrar componentes heredados o bibliotecas de terceros sin alterar su código fuente. Adapter puede implementarse mediante la composición, donde el adaptador contiene una instancia de la clase a adaptar, o mediante la herencia, extendiendo la funcionalidad de una clase existente para hacerla compatible con una nueva interfaz. Gracias a esta flexibilidad, el patrón Adapter permite la interoperabilidad entre sistemas previamente incompatibles, haciendo posible la integración de software de distintos orígenes sin necesidad de cambios en su código base.

Patrón Bridge (Puente)

El patrón de diseño Bridge es una solución estructural que desacopla una abstracción de su implementación, permitiendo que ambas evolucionen de manera independiente sin generar dependencias rígidas entre ellas. Su propósito es evitar el crecimiento exponencial de clases cuando existen múltiples dimensiones de variabilidad dentro de un sistema. Para lograrlo, divide el sistema en dos jerarquías interconectadas: una para la abstracción y otra para la implementación. De este modo, se facilita la expansión y modificación del código sin afectar la estructura general, promoviendo el principio de abierto/cerrado y mejorando la reutilización del software.

Patrón Composite (Objeto compuesto)

El patrón de diseño Composite permite componer objetos en estructuras jerárquicas de árbol, de modo que los elementos individuales y las composiciones de elementos puedan ser tratados de manera uniforme. Su principal objetivo es proporcionar una interfaz común que simplifique la manipulación de conjuntos de objetos, sin necesidad de distinguir entre elementos simples y compuestos. Esta propiedad lo hace particularmente útil cuando se trabaja con estructuras recursivas en las que cada nodo puede ser un objeto independiente o contener subelementos. Al aplicar este patrón, se logra una mayor coherencia en el diseño, facilitando la gestión y modificación de estructuras de datos complejas.



Patrón Decorator (Envoltorio)

El patrón de diseño Decorator proporciona una forma flexible de añadir funcionalidades a un objeto de manera dinámica sin alterar su estructura ni modificar directamente su código fuente. A diferencia de la herencia, que puede generar una proliferación de clases, el patrón Decorator se basa en la composición de objetos, permitiendo envolver un objeto base con múltiples capas de funcionalidad adicional. Cada decorador implementa la misma interfaz que el objeto que extiende, garantizando que puedan apilarse de manera modular y ofrecer una gran versatilidad en la personalización del comportamiento de los objetos en tiempo de ejecución.

Patrón Facade (Fachada)

El patrón de diseño Facade simplifica la interacción con sistemas complejos proporcionando una interfaz de alto nivel que oculta la implementación subyacente. Su principal beneficio es la reducción de la dependencia entre los clientes y los múltiples subsistemas internos, facilitando la integración y el mantenimiento del código. Al centralizar el acceso a un conjunto de funcionalidades, se mejora la organización del sistema y se minimiza la cantidad de conocimiento que un cliente debe tener sobre la estructura interna de los componentes. Este patrón es especialmente útil cuando se trabaja con bibliotecas de terceros o arquitecturas modulares en las que se busca simplificar la interacción con distintos módulos.

Patrón Flyweight (Peso ligero)

El patrón de diseño Flyweight se enfoca en optimizar el uso de memoria y mejorar la eficiencia de los sistemas que manejan grandes cantidades de objetos similares. Su estrategia consiste en compartir instancias de objetos con información común, separando los datos invariantes de aquellos que son específicos de cada instancia. Esto permite reducir el consumo de memoria y mejorar el rendimiento en aplicaciones donde se crean múltiples objetos con atributos repetitivos. Flyweight es ideal en entornos donde se deben manejar grandes volúmenes de datos repetitivos, como sistemas de representación gráfica, motores de videojuegos o bases de datos con elementos altamente redundantes.

Patrón Proxy

El patrón de diseño Proxy actúa como un intermediario entre un cliente y un objeto real, permitiendo controlar el acceso a este último. Se emplea para añadir funcionalidades adicionales como la carga diferida (lazy initialization), el control de acceso y la monitorización del uso de recursos. Al delegar la interacción con el objeto real a un Proxy, se pueden implementar estrategias de optimización sin modificar la estructura original del objeto subyacente. Este patrón es ampliamente utilizado en sistemas distribuidos, seguridad de datos y gestión de conexiones remotas, donde es fundamental regular la interacción con componentes críticos o costosos en términos de procesamiento.



5.3. Patrones GOF de Comportamiento

Estos patrones ofrecen soluciones respecto a la interacción y responsabilidades entre clases y objetos, así como los algoritmos que encapsulan.

Son los siguientes tipos:

- **Chain of Responsibility** (Cadena de responsabilidad):

Permite establecer la línea que deben llevar los mensajes para que los objetos realicen la tarea indicada.

- **Command** (Orden):

Encapsula una operación en un objeto, permitiendo ejecutar dicha operación sin necesidad de conocer el contenido de la misma.

- **Interpreter** (Intérprete):

Dado un lenguaje, define una gramática para dicho lenguaje, así como las herramientas necesarias para interpretarlo.

- **Iterator** (Iterador):

Permite realizar recorridos sobre objetos compuestos independientemente de la implementación de estos.

- **Mediator** (Mediador):

Define un objeto que coordine la comunicación entre objetos de distintas clases, pero que funcionan como un conjunto.

- **Memento** (Recuerdo):

Permite volver a estados anteriores del sistema.

- **Observer** (Observador):

Define una dependencia de uno-a-muchos entre objetos, de forma que cuando un objeto cambie de estado se notifique y actualicen automáticamente todos los objetos que dependen de él.

- **State** (Estado):

Permite que un objeto modifique su comportamiento cada vez que cambie su estado interno.

- **Strategy** (Estrategia):

Permite disponer de varios métodos para resolver un problema y elegir cuál utilizar en tiempo de ejecución.



- **Template Method** (Método plantilla):

Define en una operación el esqueleto de un algoritmo, delegando en las subclases algunos de sus pasos, esto permite que las subclases redefinan ciertos pasos de un algoritmo sin cambiar su estructura.

- **Visitor** (Visitante):

Permite definir nuevas operaciones sobre una jerarquía de clases sin modificar las clases sobre las que opera.



+Info

Términos a conocer:

- **Signatura:** operación realizada por un objeto, lo que se toma como parámetros y lo devuelve.
- **Interfaz de objeto:** el conjunto de todas las signaturas definidas por las operaciones de un objeto.
- **Ligadura dinámica:** es la asociación en tiempo de ejecución de una petición a un objeto, donde la operación que se realiza depende de la petición y del objeto.

6. GRASP (General Responsibility Assignment Software Patterns)

GRASP (General Responsibility Assignment Software Patterns) es un conjunto de guías y principios que proporciona directrices para asignar responsabilidades de manera efectiva en el diseño orientado a objetos. Craig Larman, prestigioso experto en el campo de la POO popularizó estos principios para ayudar a los diseñadores a asignar responsabilidades a clases y objetos de forma que se promueva un diseño robusto, mantenible y con bajo acoplamiento.



+ Importante

En GRASP (y en el diseño de software en general), el dominio hace referencia al ámbito del problema real que el software busca resolver. No se trata del software en sí mismo, sino de la representación conceptual del mundo real en el que ese software va a operar.

El dominio incluye las entidades relevantes (personas, objetos, documentos, recursos), las reglas de negocio que gobiernan su comportamiento, los procesos que se llevan a cabo y los conceptos clave que definen cómo funciona ese entorno.

Muchos principios -como Information Expert o Creator- dependen de conocer qué objetos existen en el dominio y qué responsabilidades naturales les corresponden

Los principios GRASP incluyen:

- **Controlador o Controller:** se encarga de recibir los eventos del sistema y coordinar su manejo, delegando la ejecución de las tareas en otras clases especializadas. Si durante el diseño se identifica que no hay ninguna clase adecuada para asumir la responsabilidad, esto puede indicar la necesidad de crear una nueva clase que la gestione.
- **Creador o Creator:** este principio ayuda a decidir qué clase debe ser responsable de crear instancias de otra clase. Una clase se considera candidata a ser creadora si compone o agrega a la otra, si la utiliza estrechamente, si la registra o si posee los datos necesarios para inicializarla. De este modo, se refuerza la coherencia entre las relaciones del diseño y las responsabilidades de creación de objetos.
- **Experto en Información o Information Expert:** este principio asigna la responsabilidad de realizar una tarea a la clase que dispone de la información necesaria para llevarla a cabo. De este modo, la lógica se concentra en los objetos que ya poseen los datos, lo que favorece la cohesión y reduce el acoplamiento entre clases.
- **Bajo Acoplamiento o Low Coupling:** principio que busca minimizar las dependencias entre clases para reducir el impacto de los cambios y facilitar la reutilización y las pruebas. Se promueve que las clases colaboren solo con sus colaboradores necesarios (evitando cadenas de llamadas profundas) y que dependan de abstracciones en lugar de implementaciones concretas.
- **Alta cohesión o high cohesion:** este principio promueve que cada clase tenga responsabilidades bien delimitadas y relacionadas entre sí, enfocadas en un único propósito. De este modo, todos los métodos y atributos colaboran en torno a un objetivo común, lo que favorece la claridad del diseño, facilita el mantenimiento y reduce la complejidad del sistema.



- **Polimorfismo o polymorphism:** principio que propone que un mismo mensaje u operación pueda dar lugar a diferentes comportamientos según el objeto que lo reciba o los tipos de datos sobre los que actúe. Puede manifestarse como polimorfismo paramétrico, en el que clases y métodos trabajan sobre tipos genéricos en lugar de concretos, o como polimorfismo de subtipos, donde las subclases sobrescriben métodos de su superclase para dar implementaciones específicas.
- **Fabricación Pura o Pure Fabrication:** este patrón GRASP se utiliza cuando una responsabilidad necesaria no encaja de forma natural en ninguna clase del dominio. En lugar de forzar su inclusión -lo que violaría principios como la alta cohesión o el bajo acoplamiento-, se crea una clase artificial que asuma esa responsabilidad. Estas clases no representan conceptos del dominio, sino que suelen encargarse de aspectos transversales o técnicos, como la persistencia, el registro de actividad (logging), la seguridad o las validaciones complejas, manteniendo así el modelo de dominio limpio y el diseño modular y flexible.
- **Indirection:** tiene como objetivo reducir el acoplamiento directo entre clases, introduciendo un objeto intermediario que gestione la comunicación entre ellas. De este modo, los componentes no dependen directamente unos de otros, lo que aumenta la flexibilidad y facilita el mantenimiento. Ejemplos habituales de indirection son los controladores, los manejadores de eventos o los repositorios.
- **Protected Variations:** consiste en identificar los puntos del sistema que son más propensos a cambiar -por ejemplo, reglas de negocio, tecnologías externas o dependencias volátiles- y encapsularlos detrás de una interfaz estable. De este modo, se protege al resto del sistema de los efectos de posibles modificaciones futuras, logrando mayor flexibilidad y menor riesgo ante cambios inevitables.

7. Modelado de aplicaciones: UML

UML, siglas del inglés, Unified Modeling Language o en castellano lenguaje unificado de modelado, es un lenguaje gráfico de modelado de sistemas de software, para describirlo, diseñarlo y documentarlo.



Fuente: https://commons.wikimedia.org/wiki/File:UML_logo.svg



UML es el lenguaje de modelado más conocido y utilizado y está respaldado por el Object Management Group (OMG), que es un consorcio internacional sin ánimo de lucro y de membresía abierta para estándares tecnológicos. Los estándares de OMG son promovidos por proveedores, usuarios finales, instituciones académicas y agencias gubernamentales.

UML es un estándar de modelado de OMG, es un lenguaje estándar para especificar, para describir un plano de un sistema de software.

UML se creó para forjar un lenguaje de modelado visual común para la arquitectura, el diseño y la implementación de sistemas de software complejos. De hecho, se ha convertido en el estándar para modelar aplicaciones software y su popularidad en el modelado no para de crecer en otros dominios.

Los diagramas UML describen los límites, la estructura y el comportamiento del sistema y los objetos que contiene.

UML no es un lenguaje de programación, pero existen herramientas que se pueden usar para generar código en diversos lenguajes a partir de UML

Los creadores originales de UML son 3: Jim Rumbaugh, Grady Booch e Ivar Jacobson.



+ Info

El UML es popular entre programadores, pero no suele ser usado por desarrolladores de bases de datos.

Esto se debe a que los creadores de UML no lo enfocaron para su uso en bases de datos.

Objetivos de UML

Según OMG los objetivos de UML son:

- Expresar un diseño utilizando elementos gráficos, especificando las características del sistema.
- Brindar a arquitectos de sistemas, ingenieros y desarrolladores de software las herramientas para el análisis, el diseño y la implementación de sistemas basados en software, así como para el modelado de procesos de negocios y similares.



- Hacer progresar el estado de la industria permitiendo la interoperabilidad de herramientas de modelado visual de objetos.
- No obstante, para habilitar un intercambio significativo de información de modelos entre herramientas, se requiere de un acuerdo con respecto a la semántica y notación.

Requisitos de UML

UML cumple con los siguientes requisitos:

- Establecer una definición formal de un metamodelo común basado en el estándar MOF (meta-object facility) que especifique la sintaxis abstracta del UML.

La sintaxis abstracta define el conjunto de conceptos de modelado UML, sus atributos y sus relaciones, así como las reglas de combinación de estos conceptos para construir modelos UML parciales o completos.

- Brindar una explicación detallada de la semántica de cada concepto de modelado UML.

La semántica define, de manera independiente a la tecnología, cómo los conceptos UML se habrán de desarrollar por las computadoras.

- Especificar los elementos de notación de lectura humana para representar los conceptos individuales de modelado UML, así como las reglas para combinarlos en una variedad de diferentes tipos de diagramas que corresponden a diferentes aspectos de los sistemas modelados.
- Definir formas que permitan hacer que las herramientas UML cumplan con esta especificación.

Esto se apoya (en una especificación independiente) con una especificación basada en XML de formatos de intercambio de modelos correspondientes (XML) que deben ser concretados por herramientas compatibles.

Mecanismos Comunes

Son 4 mecanismos que hacen que facilitan la construcción de bloques UML:

- **Especificaciones:** detrás de cada parte de la notación gráfica se indica textualmente la sintaxis y semántica de ese bloque.
- **Adornos:** casi todos los elementos tienen una notación gráfica, para facilitar el entendimiento de forma visual.
- **Divisiones Comunes.**
- **Mecanismos de extensión** (de extensibilidad).

Para poder expresar todos los matices posibles de todos los modelos en todos los dominios y en todos los momentos, y adaptar UML de manera controlada a las necesidades de nuevas tecnologías, UML proporciona tres mecanismos de extensión.



- Los mecanismos de extensión de UML son:

- **Estereotipos.**

Permite crear nuevos tipos de bloques de construcción. Derivan de los existentes, pero son específicos a un problema.

En lenguajes como Java o C++, es necesario modelar las excepciones, que son simplemente clases, que normalmente sólo se pueden ser lanzadas y capturadas. Para poder modelar las excepciones se puede crear un estereotipo de una clase.

- **Valores etiquetados.**

Permite añadir nueva información en la especificación de un elemento.

- **Restricciones.**

Permite añadir a un bloque de construcción de UML, nuevas reglas o modificar las existentes.

Reglas de UML

Hay una serie de normas para lograr que un modelo esté bien formado, son las siguientes reglas semánticas:

- **Nombres:** cómo llamar a los elementos, relaciones y diagramas.
- **Ámbito:** se indica el contexto que da a un nombre un significado específico.
- **Visibilidad:** hace referencia a cuantos y como los nombres pueden ser vistos y utilizados por otros.
- **Integridad:** forma adecuada de que los elementos se relacionan.
- **Ejecución:** el significado de simular (correr) un modelo dinámico.

Versiones

Estas son las versiones publicadas de UML:

- UML 1.1 (Noviembre de 1997)
- UML 1.3 (Marzo de 2000)
- UML 1.4 (Septiembre de 2001)
- UML 1.5 (Marzo de 2003)



- UML 1.4.2 (Enero de 2005)
- UML 2.0 (Octubre de 2005)
- UML 2.1 (Abril de 2006)
- UML 2.1.1 (Febrero de 2007)
- UML 2.1.2 (Noviembre de 2007)
- UML 2.2 (Febrero de 2009)
- UML 2.3 (Mayo de 2010)
- UML 2.4.1 (Agosto de 2011)
- UML 2.5 (Junio de 2015)
- UML 2.5.1 (Diciembre de 2017) Es la última versión.

7.1. Bloques de construcción de UML

Un modelo UML se compone de diferentes bloques de construcción que son:

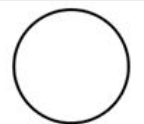
- Elementos.
- Relaciones.

7.1.1. Elementos

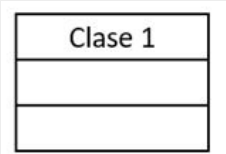
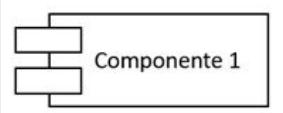
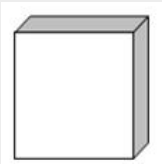
Hay 3 clases de elementos: estructurales, comportamiento y agrupación (también se puede considerar anotación como elemento).

- **Elementos estructurales:**

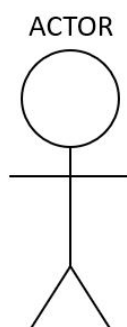
Vamos a indicar los básicos que se pueden incluir en el modelo UML.

Nombre y Símbolo	Descripción
Interfaz 	Colección de operaciones que especifican un servicio de una clase o componente. Representa el comportamiento completo de una clase o componente.



Nombre y Símbolo	Descripción
Colaboración 	Interacción. Roles y otros elementos que colaboran para proporcionar un comportamiento cooperativo mayor que la suma de los comportamientos de sus elementos. Las colaboraciones representan la implementación de patrones que forman un sistema.
Caso de Uso 	Descripción de secuencias de acciones que un sistema ejecuta y que produce un resultado. Se utiliza para estructurar el comportamiento en un modelo.
Clase Activa 	Objeto que tiene uno o más procesos o hilos de ejecución y por lo tanto pueden dar origen a actividades de control. Se diferencian de las clases en que sus objetos representan a elementos cuyo comportamiento es concurrente con otros elementos.
Componente 	Representa el empaquetamiento físico de diferentes elementos lógicos (por ejemplo, clases, interfaces y colaboraciones).
Nodo 	Es un elemento físico que existe en tiempo de ejecución y representa un recurso computacional que dispone de memoria y con frecuencia capacidad de procesamiento.

Existen variaciones de estos seis elementos, tales como actores, señales, procesos, hilos y aplicaciones, documentos, archivos, bibliotecas, páginas y tablas.





- **Los elementos de comportamiento:**

Son las partes dinámicas (verbos) de los modelos UML que representan comportamiento en el tiempo y en el espacio.

Están conectados normalmente a diversos elementos estructurales, principalmente clases, colaboraciones y objetos, y normalmente están conectados a diversos elementos estructurales, principalmente clases, colaboraciones y objetos. Son:

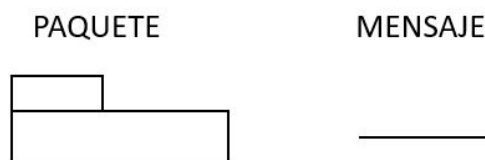
- **Interacción:** Comportamiento que comprende un conjunto de mensajes intercambiados entre un conjunto de objetos, en un contexto determinado para alcanzar un propósito específico.
- **Máquina de Estados:** Comportamiento que especifica las secuencias de estados por las que pasa un objeto o una interacción en respuesta a eventos.

- **Los Elementos de agrupación:**

Son las partes organizativas de los modelos UML, las cajas en las que pueden descomponerse un modelo.

- **Paquete:** son los elementos de agrupación básicos con los cuales se puede organizar un modelo UML. Se pueden agrupar los elementos estructurales, de comportamiento e incluso otros elementos de agrupación.

Hay variaciones, como los framework, los modelos y los subsistemas.



7.1.2. Relaciones

Relacionan los elementos entre sí, y hay 4 tipos diferentes:

- **Dependencias.**

Es una relación semántica entre dos elementos, en la cual un cambio a un elemento puede afectar el significado de otro elemento.

Hay tipos de dependencias predefinidas que se indican para casos de uso mediante extend o include.

- extend (la extensión).

Un caso de uso puede extenderse a otro. El comportamiento del caso de extensión se puede insertar en el caso de uso extendido en determinadas condiciones.

Se indica mediante la notación de una flecha rayada desde el caso de uso extensión hasta el extendido junto a la etiqueta extend.



- include.

Un caso de uso puede incluir a otro, normalmente el resultado del primero va a depender del caso de uso incluido.

Se indica mediante la notación de una flecha rayada desde el caso de uso que lo incluye hasta el que es incluido junto a la etiqueta include.

- **Asociaciones.**

Es una relación estructural entre dos elementos, que describen las conexiones entre ellos. Representan una relación estructural entre un todo y sus partes, agregación o composición.

- **Generalizaciones.**

Es una relación entre elemento padre (más general) y elemento hijo (más específico). Se relaciona con el concepto de herencia en la POO.

- **Implementación.**

Es una relación en la que el elemento hijo realiza las acciones indicadas por el padre.

7.2. Diagramas UML

Un diagrama es la representación gráfica de un conjunto de elementos y de las relaciones entre ellos. Constituye una proyección parcial del sistema, que ofrece una vista resumida de sus componentes y permite analizarlo desde diferentes perspectivas.

Representa una vista resumida de los elementos que constituyen un sistema, lo que permite visualizarlo desde diferentes perspectivas.

UML utiliza dos tipos básicos de diagramas:

- **Diagramas de Estructura o Estructurales.**

Los diagramas estructurales representan los aspectos estáticos de un sistema, es decir, su organización interna, los elementos que lo componen y las relaciones entre ellos. Permiten mostrar la arquitectura del sistema desde un punto de vista estable, independiente de su ejecución.

- Diagrama de clases.
- Diagrama de objetos.
- Diagrama de paquetes.
- Diagrama de componentes.



- Diagrama de despliegue.
- Diagrama de estructura compuesta.
- Diagrama de perfiles.

- **De comportamiento.**

Representan los aspectos dinámicos de un sistema, es decir, cómo interactúan sus elementos y cómo evolucionan a lo largo del tiempo. Permiten describir casos de uso, flujos de actividades, estados de los objetos y la forma en que los elementos colaboran entre sí.

- Diagramas de casos de uso.
- Diagrama de actividades.
- Diagrama de máquina de estados.
- Diagrama de interacción:
 - » Diagrama de secuencia.
 - » Diagrama de comunicación (colaboración).
 - » Diagrama de panoramas de interacción.
 - » Diagrama de temporización.

7.2.1. Diagramas UML Estructurales

Diagramas que representan aspectos estáticos o estructurales de un sistema.

Diagrama de Clases

Es el diagrama UML más usado, y la base principal de toda solución orientada a objetos. Es la agrupación de clases con las relaciones entre ellas, que son indicadas mediante flechas.

Muestra las clases dentro de un sistema, atributos, operaciones y la relación entre cada clase. Son utilizadas durante el proceso de análisis y diseño.

La forma de representación es: una clase tiene tres partes áreas, se representan con rectángulos divididos en tres:

- La superior contiene el nombre de la clase.
- La central contiene los atributos.
- La inferior las acciones (operaciones o métodos). Son actividades o verbos que se pueden realizar con o para un objeto.



Convenciones habituales de notación: si el nombre de un método es una sola palabra se escribe en minúsculas; si son varias palabras, la primera va en minúsculas y las siguientes con la inicial en mayúscula (notación camelCase, sin espacios).

Un objeto se representa de manera similar a una clase, pero en la parte superior se indica el nombre del objeto junto con el de la clase, y se subraya para marcar que es una instancia.

Las **asociaciones** se representan mediante una línea que une las clases y que puede llevar símbolos que indican características de la asociación. La **agregación** se representa mediante un diamante colocado en el extremo que representa el "todo". Si participan más de dos clases, estas se unen con una línea al diamante central. También se puede especificar un nombre de rol para indicar el papel que tiene una clase en una asociación.

Diagrama de Objetos

Muestra la relación entre objetos por medio de ejemplos del mundo real e ilustra cómo se verá un sistema en un momento dado.

Dado que los datos están disponibles dentro de los objetos, estos pueden usarse para clarificar relaciones entre ellos.

Se utilizan en la etapa de análisis y diseño y sirven como "instantánea" del sistema en ejecución.

Diagrama de Paquetes

El diagrama de paquetes muestra la división de un sistema en agrupaciones lógicas de elementos UML, **habitualmente clases**, aunque también puede contener casos de uso, componentes u otros elementos del modelo. Representa las dependencias entre paquetes y cómo se organizan para revelar la arquitectura del sistema.

Hay dos tipos especiales de dependencias que se definen entre paquetes:

- La importación de paquetes.
- La fusión de paquetes.

Los paquetes pueden representar los diferentes niveles de un sistema para revelar la arquitectura. También se pueden marcar las dependencias de paquetes para mostrar el mecanismo de comunicación entre niveles.

Diagrama de componentes (o despliegue)

Representa cómo un sistema de software se divide en componentes y la dependencia que hay entre ellos. Se indicarán librerías, tablas, archivos, ejecutables, documentos, etc.

Se utiliza para modelar la organización de los artefactos software y las dependencias entre sus módulos. Los componentes se comunican por medio de interfaces, y con este diagrama se muestra qué partes pueden compartirse entre diferentes secciones de un sistema o incluso entre sistemas distintos.



Diagrama de despliegue

Muestra la arquitectura física del sistema como el despliegue de los artefactos de software en los nodos de hardware. Es útil cuando se implementa una solución de software en múltiples máquinas con distintas configuraciones.

Se utiliza para modelar la disposición física de los elementos de software en nodos (servidores, estaciones, dispositivos, etc.) y permite visualizar la arquitectura en entornos complejos.

Diagrama de estructura compuesta

Se usa para mostrar la estructura interna de una clase o de un componente. Permite detallar cómo se organizan sus partes, puertos y conectores internos, reflejando la colaboración entre sus elementos constitutivos.

Diagrama de perfiles

Es un diagrama UML auxiliar, que sirve para definir valores etiquetados, estereotipos personalizados y restricciones.

Los perfiles permiten la adaptación de UML para diferentes plataformas o dominios, como por ejemplo Java o .NET.

En este tipo de diagramas los elementos gráficos son: estereotipo, extensión, metadatos, referencia, perfil y aplicación de perfil. Fue el último tipo de diagrama añadido por UML 2.0.

7.2.2. Diagramas UML de Comportamiento

Los diagramas de comportamiento representan los aspectos dinámicos de un sistema, es decir, cómo evolucionan los elementos con el tiempo, cómo interactúan entre sí y qué funcionalidades proporcionan. Permiten modelar procesos, interacciones, estados y escenarios de uso, ofreciendo una visión de cómo el sistema se comporta durante su ejecución.

Diagramas de casos de uso

Representa una funcionalidad particular de un sistema. Se crea para ilustrar cómo se relacionan las funcionalidades con sus controladores (actores) internos y externos.

No deben ser excesivamente genéricos ni demasiado específicos.

Existe una notación gráfica llamada modelo de casos de uso que no debe confundirse con la descripción textual de los casos de uso (para la cual no existe un formato único estandarizado).



Los elementos que aparecen en estos diagramas son:

- **Actores:** representan usuarios u otros sistemas que interactúan con el sistema.
- **Casos de uso:** representan la descripción de las interacciones que se producen entre un actor y el sistema cuando el actor utiliza el sistema para algo concreto. El nombre del caso de uso debe describir claramente la funcionalidad.

Relaciones principales entre casos de uso:

- **Include:** un caso de uso incluye siempre la ejecución de otro.
- **Extend:** un caso de uso puede opcionalmente extender otro bajo ciertas condiciones.
- **Generalization:** existe una relación de especialización entre casos de uso. Se representa mediante una línea sólida terminada en un triángulo, desde el caso de uso especializado hacia el general.

Dada su relevancia como herramienta central de análisis y captura de requisitos, en este temario se desarrollan en el próximo epígrafe.

Diagrama de actividades

Indica flujos de trabajo de negocios u operativos representados gráficamente para mostrar la actividad de alguna parte o componente del sistema. Refleja, por tanto, el flujo de control general.

Los diagramas de actividades se utilizan como una alternativa a los diagramas de máquina de estados para modelar procesos más orientados al flujo.

Diagrama de Máquina de Estados

Son similares a los diagramas de actividades, pero se centran en describir el comportamiento de objetos que cambian en función de su estado actual.

Notación:

- Círculo lleno: estado inicial.
- Círculo hueco con otro círculo lleno en su interior: estado final.
- Rectángulo redondeado: representa un estado, con el nombre en la parte superior y las actividades que se realizan dentro del estado debajo de una línea de separación.
- Flecha: transición entre estados. Puede incluir:
 - Evento que causa la transición.
 - Condición de guarda entre corchetes [].
 - Acción asociada a la transición, indicada tras una barra «/».



- Ejemplo: evento[condición]/acción.
- Línea horizontal gruesa: puede representar una unión (varias entradas y una salida) o una bifurcación (una entrada y varias salidas).

Diagramas de Interacción

Los diagramas de interacción son un subgrupo dentro de los diagramas de comportamiento. Describen cómo los objetos colaboran entre sí a través del intercambio de mensajes.

- **Diagrama de Secuencia.**

Muestra cómo los objetos interactúan entre sí y el orden en el que ocurren los mensajes. Representa interacciones para un escenario concreto y hace hincapié en la dimensión temporal.

- **Diagrama de Comunicación (o de Colaboración).**

Son similares a los diagramas de secuencia, pero se centran en las relaciones entre los objetos y los mensajes que se intercambian.

No muestran explícitamente el tiempo como dimensión aparte, por lo que es necesario numerar los mensajes para indicar su orden.

Sirven para mostrar cómo las instancias específicas de clases trabajan juntas para conseguir un objetivo común.

Implementan las asociaciones del diagrama de clases mediante enlaces (mensajes pasados de un objeto a otro).

- **Diagrama de Panorama de Interacciones (o Visión General de Interacciones).**

Permite ver, de manera resumida, cómo se relacionan varias interacciones y en qué orden ocurren. Proporciona una visión global de la secuencia de interacciones.

- **Diagrama de Temporización.**

Al igual que los diagramas de secuencia, representan el comportamiento de los objetos en un periodo de tiempo.

- Si hay un solo objeto, el diagrama es simple.
- Si intervienen varios objetos, se muestran sus interacciones durante ese lapso temporal concreto.



7.2.2.1. Diagramas de casos de uso

Como ya hemos visto, los diagramas de casos de uso forman parte de los diagramas de comportamiento de UML, sin embargo, añadimos aquí este epígrafe independiente debido a su especial relevancia. Estos diagramas no solo se utilizan para modelar el comportamiento del sistema, sino que además cumplen una función fundamental en la captura y comunicación de requisitos funcionales entre perfiles técnicos y no técnicos. Por esta razón, su estudio se desarrolla con mayor detalle que el de otros diagramas, manteniendo así la coherencia con la práctica habitual en proyectos de análisis y diseño de software.

El diagrama de casos de uso es un diagrama UML de comportamiento que describe, desde la perspectiva externa, qué funciones debe ofrecer un sistema a quienes interactúan con él. Su finalidad es capturar requisitos funcionales de alto nivel de forma comprensible para perfiles técnicos y no técnicos. El diagrama muestra actores externos, los servicios u objetivos que persiguen dentro del sistema y el límite que separa el interior del sistema de su entorno. Este enfoque responde a la pregunta qué hace el sistema y evita deliberadamente detallar cómo lo hace, reservando ese nivel para otros diagramas o especificaciones.

Definición y objetivo

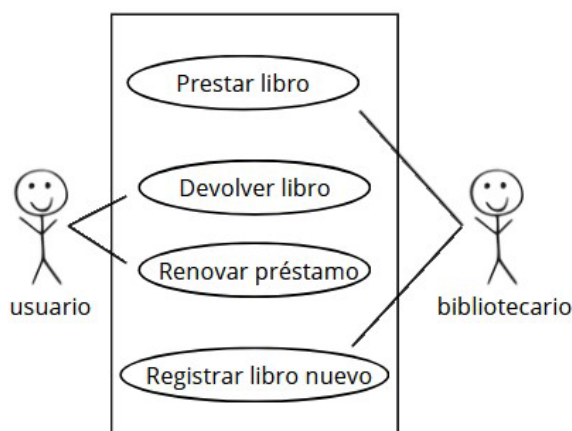
Un caso de uso es una secuencia de interacciones entre un actor y el sistema que produce un resultado con valor para ese actor. El objetivo del diagrama es identificar las capacidades visibles del sistema, acotar su alcance y establecer un lenguaje común con los interesados. La utilidad principal reside en transformar necesidades del negocio en funcionalidades verificables, que después podrán descomponerse en requisitos, reglas y pruebas.

Elementos: actores, casos de uso y límite del sistema

Un actor es cualquier entidad externa que interactúa con el sistema: personas, otros sistemas u organizaciones. Se modela por su rol, no por individuos concretos; por ejemplo, ciudadano, empleado público o sistema de pagos. Un caso de uso representa una funcionalidad ofrecida por el sistema que satisface un objetivo del actor; conviene nombrarlo con un verbo en infinitivo y un complemento que denote el valor aportado, como presentar solicitud o emitir certificado. El límite del sistema es un rectángulo que encierra todos los casos de uso y delimita lo que es responsabilidad del sistema frente al entorno. Los actores siempre quedan fuera de ese límite; las interacciones cruzan la frontera mediante asociaciones.

Relaciones: asociación, include, extend y generalización

La asociación conecta un actor con los casos de uso en los que participa. La relación include indica que un caso de uso incorpora obligatoriamente el comportamiento de otro, útil para factorizar pasos comunes como autenticar usuario o validar datos. La relación extend expresa comportamiento opcional que se activa bajo ciertas condiciones, por ejemplo, un proceso de solicitud que puede extenderse con requerir subsanación si falta documentación. La generalización permite especializar actores o casos de uso cuando comparten parte del comportamiento; por ejemplo, empleado público puede especializarse en tramitador y supervisor si ambos comparten casos base con variaciones.



Notación y reglas básicas

Los actores se dibujan como figuras estilizadas fuera del rectángulo del sistema. Los casos de uso se representan como óvalos en el interior del sistema y se nombran con claridad y consistencia terminológica. Las asociaciones se trazan como líneas; include y extend se indican con estereotipos en la línea dirigida hacia el caso incluido o extendido. Es buena práctica mantener el diagrama en un nivel alto, evitando saturarlo con demasiados casos o relaciones. Cuando el dominio es amplio, es preferible dividir en vistas temáticas y complementar con descripciones textuales para cada caso de uso. El diagrama no detalla reglas de negocio ni flujos internos; esos elementos se documentan en la especificación del caso de uso y se refinan con otros diagramas, como de actividades, de secuencia o de estados.

Especificación textual del caso de uso

Cada caso de uso debe acompañarse de una breve especificación que permita entender el flujo principal y las variantes. Una plantilla sencilla y efectiva incluye nombre, objetivo, actor principal, interesados y necesidades, precondiciones, postcondiciones, flujo básico y flujos alternativos o de excepción. Las precondiciones describen lo que debe cumplirse antes de iniciar el caso, como estar autenticado o disponer de un expediente en estado borrador. Las postcondiciones fijan el resultado observable al finalizar con éxito o tras una excepción gestionada, como expediente registrado y acuse emitido. El flujo básico narra, en pasos numerados y en tiempo presente, la interacción ideal desde el inicio hasta el resultado. Los flujos alternativos describen desvíos condicionados, por ejemplo, documentación incompleta o pago rechazado, y su reencaje en el flujo principal. Esta especificación se utiliza después como base para pruebas de aceptación.



+ Info

Un caso de uso describe qué espera el usuario del sistema. Una prueba de aceptación verifica que el sistema realmente cumple ese comportamiento.



Buenas prácticas y errores frecuentes

Es recomendable nombrar los casos con verbos orientados a valor y lenguaje de negocio, agruparlos por áreas funcionales y mantener consistencia terminológica con el resto del temario. Conviene factorizar pasos repetidos mediante `include` y reservar `extend` para comportamientos realmente opcionales y condicionados. Un error común es confundir pantallas o acciones técnicas con casos de uso; casuística como cargar formulario o pulsar botón pertenece al diseño de interfaz, no al nivel de requisitos. Otro error frecuente es saturar un único diagrama con demasiada información; es preferible varias vistas cohesivas y la especificación textual adjunta.

Ejemplo orientado a Administraciones Públicas

Se considera un sistema de tramitación de expedientes. Los actores son ciudadano, empleado público y sistema de notificaciones. Dentro del sistema se modelan casos como presentar solicitud, aportar documentación, consultar estado, revisar expediente y emitir resolución. El ciudadano se asocia a presentar solicitud, aportar documentación y consultar estado. El empleado público se asocia a revisar expediente y emitir resolución. El caso de uso presentar solicitud incluye autenticar usuario y validar datos. El caso revisar expediente puede extenderse con requerir subsanación si se detectan omisiones. La postcondición de emitir resolución establece expediente resuelto y notificación emitida al ciudadano. La interacción con el sistema de notificaciones se modela como asociación desde emitir resolución hacia ese actor externo, dejando claro que el envío efectivo es responsabilidad del sistema externo.

Relación con otros diagramas y con el resto del temario

El diagrama de casos de uso se complementa con diagramas de actividades para detallar flujos de trabajo, de secuencia para describir intercambios entre objetos o servicios y de estados cuando el ciclo de vida de una entidad requiere precisión. En el temario, este epígrafe enlaza con los contenidos de metodologías de desarrollo, donde los casos de uso se emplean para capturar requisitos y derivar pruebas, y se contrasta con los diagramas de flujo de datos del diseño conceptual, que modelan el recorrido y transformación de la información. La separación conceptual es útil: el caso de uso se centra en objetivos del actor y servicios visibles; el diagrama de flujo de datos se centra en entradas, transformaciones y salidas de datos.

7.2.3. Manual de Modelado UML

Diagrama de Casos de Uso

¿Qué debe hacer el sistema?

Para entender como funciona nuestro sistema, lo conveniente es iniciar el proceso de modelado capturando los requisitos funcionales desde la perspectiva del usuario. El objetivo principal es **identificar actores, sus interacciones con el sistema** y establecer el alcance funcional completo.



Diagramas de Interacción

¿Cómo deben interactuar los elementos del sistema?

Estos diagramas (secuencia y comunicación) detallan el flujo de interacciones para cada caso de uso crítico. Permiten identificar objetos participantes, mensajes intercambiados y descubren operaciones preliminares. Sirven como puente esencial entre los requisitos y el diseño detallado. El cometido esencial es el **descubrir las operaciones** que posteriormente formarán parte del Diagrama de Clases.

Diagrama de Actividades

¿Cómo es el flujo del proceso?

Complementa el modelado de comportamiento para flujos de trabajo complejos o procesos empresariales. Es particularmente útil cuando se necesita visualizar el flujo de control entre actividades, decisiones paralelas y sincronización de procesos.

Diagrama de Estados

¿Cómo reacciona un objeto a eventos según su estado actual?

Se especializa en modelar el comportamiento de objetos cuyo funcionamiento depende críticamente de su estado interno. Es ideal para representar máquinas de estado finito y transiciones entre diferentes estados de un objeto o sistema.

Diagrama de Clases

¿Cuál es la composición del sistema?

Constituye el corazón del modelo estructural del sistema. Aquí se definen formalmente las clases con sus atributos, operaciones, relaciones, asociaciones y generalizaciones. Se construye progresivamente a partir de los objetos identificados en los diagramas de comportamiento.

Diagrama de Paquetes

¿Cómo se organizan y agrupan los elementos del sistema?

Organiza los elementos del diagrama de clases en grupos lógicos y módulos coherentes. Gestiona las dependencias entre componentes de alto nivel y ayuda a prevenir problemas de acoplamiento excesivo en la arquitectura del sistema.

Diagrama de Componentes

¿Cómo se empaquetan e implementan físicamente los módulos del software?

Representa la estructura física del software mostrando ejecutables, librerías y sus dependencias. Ilustra cómo se empaquetan e interrelacionan los módulos software del sistema para su implementación práctica.



Diagrama de Despliegue

¿Dónde se ejecutan los componentes?

Especifica la arquitectura hardware necesaria para el sistema. Muestra la distribución física de componentes en nodos de procesamiento, dispositivos y configuraciones de red requeridas para el entorno de producción.

Flujo de Modelado Integrado

¿Cómo se pasa de la idea al despliegue?

El proceso sigue una progresión natural que comienza con requisitos, avanza mediante casos de uso, se detalla con diagramas de interacción, formaliza con diagramas de clases, organiza con paquetes, implementa con componentes y finalmente se despliega en hardware. Este enfoque mantiene coherencia entre todas las vistas del sistema.

Ejemplo

- IDEA: "Necesito una web para vender productos".
- CASOS DE USO: Defino "Qué" debe hacer: "Realizar pedido", "Gestionar productos".
- SECUENCIA/ACTIVIDADES: Defino "Cómo" se hace: Flujo de pasos e interacciones.
- CLASES: Defino "De qué" se compone: Clases Pedido, Producto, Cliente.
- PAQUETES: Agrupo clases en módulos para organizar el código.
- COMPONENTES: Empaquito todo en archivos ejecutables (.jar, .dll).
- DESPLIEGUE: Decido "Dónde" se ejecuta: En qué servidores y con qué configuración.

8. El Proceso Racional Unificado (RUP)

Originalmente, se diseñó un proceso genérico y de dominio público, el Proceso Unificado, y una especificación más detallada, el Rational Unified Process (RUP), que se vendiera como producto independiente.

RUP, es un proceso de desarrollo de software que fue desarrollado por la empresa Rational Software, y actualmente es propiedad de IBM. RUP no es un sistema con pasos firmemente establecidos, sino un conjunto de metodologías adaptables al contexto y necesidades de cada organización.

También se conoce por este nombre al software, también desarrollado por Rational, que incluye información entrelazada de diversos artefactos y descripciones de las diversas actividades. Está incluido en el Rational Method Composer (RMC), que permite la personalización de acuerdo con las necesidades.



RUP junto con UML, constituyen la metodología estándar más utilizada para el análisis, diseño, implementación y documentación de sistemas orientados a objetos.

El ciclo de vida RUP es una implementación del desarrollo en espiral. Las tareas se organizan en fases e iteraciones.

RUP divide el proceso en cuatro fases, dentro de las cuales se realizan pocas pero grandes y formales iteraciones, en número variable según el proyecto, son:

- **Iniciación.**

Aquí, y también en la fase de elaboración, las iteraciones se enfocan hacia la comprensión del problema y la tecnología, delimitar el ámbito del proyecto, eliminar riesgos críticos, y establecer una baseline de la arquitectura.

En esta fase el hincapié se hace especialmente en actividades de requisitos y modelado del negocio.

- **Elaboración.**

Las iteraciones se centran más en el desarrollo de la baseline de la arquitectura, abarcan más los flujos de trabajo de requisitos, refinamiento del modelo de negocios, análisis y diseño.

- **Construcción.**

Se lleva a cabo la construcción del producto por medio de una serie de iteraciones.

Para cada iteración se seleccionan algunos Casos de Uso, refinando su análisis y diseño y realizando pruebas de implementación.

Se hacen iteraciones hasta que se termina la implementación del producto.

- **Transición.**

El objetivo es garantizar que se tiene un producto preparado para su uso.

Principios de desarrollo

La Filosofía del RUP está basado en los siguientes 6 principios clave:

- **Adaptar el proceso.**

El proceso tiene que adaptarse a las necesidades del cliente, por lo que es imprescindible interactuar con él.

Las características del proyecto, tamaño, alcance, etc., influirán en su diseño.



- **Equilibrar prioridades.**

En ocasiones los requisitos de los diversos participantes pueden ser diferentes, contradictorios o que generen disputa por los recursos disponibles. Hay que encontrar un equilibrio que permita satisfacer los deseos de todos, y corregir desacuerdos que puedan surgir en el futuro.

- **Demostrar valor iterativamente.**

Se realiza la entrega de los proyectos en etapas iteradas (aunque la entrega sea interna), y en cada iteración se revisa hacia dónde va la dirección del proyecto, la estabilidad y calidad, los riesgos involucrados y también se analiza la opinión de los inversores.

- **Colaboración entre equipos.**

El desarrollo de software lo realizan múltiples equipos, por lo que debe haber una comunicación fluida entre ellos que permita coordinar requisitos, desarrollo, evaluaciones, resultados, etc.

- **Enfocarse en la calidad.**

El control de calidad no debe realizarse al final de cada iteración, sino en todos los aspectos de la producción. El aseguramiento de la calidad forma parte del proceso de desarrollo y no de un grupo independiente, también es una estrategia de desarrollo de software.

- **Elevar el nivel de abstracción.**

Facilitar el proceso de desarrollo mediante diferentes herramientas.

9. Bibliografía

- JOYANES AGUILAR, L. *Fundamentos de programación*. McGraw-Hill, 2008.
- <https://www.lucidchart.com/pages/es/qu%C3%A9-es-el-lenguaje-unificado-de-modelado-uml>.
- http://ferestrepoca.github.io/paradigmas-de-programacion/poo/poo_teor%C3%ADa/2017-1POO.pdf.
- <https://es.wikipedia.org>.
- <https://es.slideshare.net/eduardolopezr/programacin-orientada-a-objetos-53132601>.
- <https://prezi.com/dhxe5b0pph3/13-relacion-entre-clases-y-objetos/>.
- <http://www.cristalab.com/tutoriales/programacion-orientada-a-objetos-asociacion-vs-composicion-c893371/>.
- <https://dile.rae.es/paradigma>.

